

NCP

A correct, lean closed-loop boundary

Engram / NEST / CREBAIN

Prisoma / Galadriel / Haldir

Source re-audit, mathematical checks, and a cross-repository plan for NCP-only runtime integration.

5 September 2026

Proposed changes. Mathematical checks executed.
Native builds, NEST runs, and integration benchmarks not executed in this review.

Contents

1. Executive decision	4
1.1 Decisions to implement	4
1.2 What this report delivers—and does not claim	4
2. Corrections and qualifications to the previous audit	5
3. Current consumer fit	6
3.1 CREBAIN	6
3.2 Prisoma	6
3.3 Galadriel	6
3.4 Haldir	7
3.5 NCP’s role	7
4. Source findings and required corrections	8
4.1 P0: loss accounting occurs after intentional coalescing	8
4.2 P0: consumption time is not original receipt time	8
4.3 P0: codec output can hide absence or ambiguous layout	8
4.4 P0 for admitted scales: small ranges collapse in interpolation	8
4.5 P0/P1: idempotency protects mutation better than outcome recovery	9
4.6 P1: bounded CUSUM work changes permitted detector behavior	9
4.7 P1: thread-local origins are not a movable object’s clock	9
4.8 P1: strict horizon behavior needs one executable definition	9
4.9 P1: exact contract verification accepts missing identity	10
4.10 P0 for secure profiles: transport identity is not established by a payload claim	10
4.11 What is worth retaining	10
5. Low-overhead implementation: remove repetition, not guarantees	10
5.1 Prepared objects and typed ownership	10
5.2 Queue and memory policy	11
5.3 Observer isolation	11
5.4 Transport selection	11
6. The NCP-only ecosystem boundary	13
6.1 Allowed topology	13
6.2 Explicit prohibitions	13
6.3 Enforceable connection policy	13
6.4 Resource and code authority during simulation creation	14
7. Execution contract: exact steps, original freshness, known outcomes	14
7.1 Prepare	14
7.2 Step	14
7.3 Local outcome state machine	15
7.4 Coupled commit is not an atomic transaction	15
7.5 Source-bound freshness without synchronized clocks	15
7.6 Cancellation, restart, and finish	16
7.7 Final-effect currentness and revocation ordering	16
8. Consumer-specific implementation requirements	16
8.1 Engram and NEST	16
8.2 CREBAIN	17
8.3 Prisoma	17
8.4 Galadriel	18
8.5 Haldir	18
9. Mathematical contracts and executable checks	19
9.1 Strict horizon count	19

9.2 Constant-work CUSUM batching	19
9.3 Exact common timebase	19
9.4 One hidden step of delay changes stability	20
9.5 Deterministic direct/NCP equivalence	20
9.6 Freshness on one receiver-owned clock	20
9.7 Closed-prefix consistency and at-most-once execution	21
9.8 Recording cannot evade conservation of backlog	21
9.9 Rate-code noise versus feedback window	21
9.10 Advisory authority is monotone	22
10. Performance claims and measurement design	22
10.1 Define the complete quantity	22
10.2 Scale the problem before optimizing	22
10.3 Paired baseline methodology	22
10.4 Qualification matrix	23
11. What to simplify—and what not to oversell	23
12. Cross-repository implementation handoff	24
12.1 Gate 0: obtain actual source identities before changing code	24
12.2 Change set A: correctness in NCP core	24
12.3 Change set B: neutral prepared execution contract	24
12.4 Change set C: local NCP binding and fast path	24
12.5 Change set D: Engram native orchestration	25
12.6 Change set E: CREBAIN endpoint	25
12.7 Change set F: Prisoma capture	25
12.8 Change set G: Galadriel and optional Haldir	25
12.9 Change set H: independent conformance, distribution, and release evidence	25
12.10 Definition of done	25
13. Prior-audit coverage and claim ledger	26
Appendix A. Seventy acceptance cases	28
A. Causality and clocks	28
B. Stream positions and delivery semantics	28
C. Stateful execution and retries	29
D. Layout, units, and missingness	29
E. NEST adapter behavior	30
F. CREBAIN and controller behavior	30
G. Prisoma and evidence integrity	30
H. Resource bounds and scheduling	31
I. Transport identity and conformance	31
J. Performance evidence and release usability	32
K. Expanded boundary, numerical, and recovery tests	32
Acceptance gate order	34
Appendix B. Execution instructions for the implementation agent	35
Appendix C. Reproducible mathematical verification program	36
Appendix D. Recorded verification output	41
Appendix E. Sources, retrieval scope, and evidence discipline	43
Retrieval limitations that must not be hidden	44

1. Executive decision

Make NCP the exclusive runtime integration boundary between Engram and CREBAIN, Prisoma, Galadriel, and an explicitly selected Haldir deployment. Engram owns NEST construction, neural execution, experiment orchestration, and closed-loop validation. Each other project retains ownership of its own state and decisions.

This is a design requirement for the implementation agent, not a description of an already delivered integration. NCP should remain a protocol and library boundary, not become a mandatory central broker, a second experiment orchestrator, or a container for arbitrary remote code.

The previous audit's central judgment stands: NCP has useful engineering foundations, but the inspected public surfaces do not establish a qualified, low-overhead native NCP NEST-CREBAIN closed loop. The candidate and consumer documentation explicitly distinguish historical wire-0.8 integrations, a blocked wire-1.0 candidate, and proposed low-overhead work. S01, S11, S13, S14, S19

The highest-value change is **a small execution contract with exact causality and recoverable outcomes**, not simply a faster serializer. An extra logical step of delay can destabilize a controller even when transport time is negligible. An uncertain simulator mutation can invalidate subsequent results even when every subsequent packet is delivered. Both statements are demonstrated under explicit assumptions in Section 9.

1.1 Decisions to implement

A single project-to-project runtime path. Replace externally exposed Engram Host API integration with actual NCP endpoints. Existing project-local simulation kernels and internal codecs may be reused behind those endpoints. Passing arbitrary legacy Host API objects through an opaque NCP payload does not satisfy this requirement.

Two execution profiles. Deterministic lockstep uses exact logical dependencies and permits bounded pausing for required evidence. Real-time sampled control uses explicit source-age, deadline, coalescing, and fallback policies. Do not advertise them as one interchangeable execution mode.

One owner per mutable backend. Only Engram's NEST owner advances its neural state. Only CREBAIN's body owner advances its simulated body. One dependent step is outstanding per coupled run unless a separately specified coupling algorithm proves another schedule appropriate.

Closed-prefix outcomes. A step is complete only when the required neural and body outcomes are known and joined. An indeterminate mutation retires the affected coupled generation; it does not trigger a blind retry or a pretend rollback.

Prepared static meaning. Bind identities, units, component order, missingness, timebase, action phase, authority mode, and resource limits once. Retain dynamic admission and final-effect checks. Do not repeatedly reconstruct the full static contract in every tick.

Optional Haldir, never ambiguous Haldir. In direct simulation mode Engram is the sole commander. In gated mode Engram sends typed intent through NCP to Haldir, Haldir is the sole commander, and CREBAIN remains the final executor. Galadriel provides assessments, not permission to actuate. Prisoma records and analyzes; it is not an implicit control-plane authority.

1.2 What this report delivers—and does not claim

This document incorporates the previous NCP audit and all 50 proposed acceptance cases, with corrections, additional findings, expanded mathematics, and a cross-repository migration plan. It is self-contained: another agent does not need the conversation to use it.

Three different kinds of checking were performed: source-symbol review, comparison against consumer contracts and NEST documentation, and executable checks of mathematical examples and abstract state models. These are not three independent full implementation audits.

Actually executed: the standard-library Python verification program reproduced in Appendix C. Its recorded output is reproduced in Appendix D. These checks establish the stated finite cases and arithmetic, not correctness of the Rust code or the real simulator stack.

Not executed: repository builds, Rust or TypeScript tests, NEST runs, cross-process integrations, failure injection into actual services, or performance benchmarks. No repository modifications were made. No measured latency improvement is claimed.

Source limitation: public source was inspected through web retrieval on September 5, 2026. An immutable, locally verified multi-repository checkout could not be obtained in this environment. References to main are therefore mutable source observations, not pinned audit artifacts. Historical commit IDs appearing in consumer documents are not relabeled as current HEAD. The implementation agent must perform the snapshot gate in Section 12 before relying on exact current-code findings.

The public Engram repository is not a sufficient implementation surface; Prisoma identifies the executable host as `sepahead/Paper2Brain`. This review did not inspect that host implementation. Galadriel’s detailed implementation was not fully retrievable during the recheck; its exact current internal schema and detector behavior remain an explicit verification task. Its proposed NCP role can nevertheless be specified without pretending that integration is already qualified. S14, S11

2. Corrections and qualifications to the previous audit

The following statements supersede any broader interpretation of the earlier report.

Earlier topic	Corrected or narrowed conclusion
“Prepare NEST once”	Keep the network and state persistent. A single Prepare across all steps is valid only for the selected, tested live-update mechanism. Some updates require a legal recalibration boundary; do not promise arbitrary SetStatus changes after preparation.
Missing NEST evidence	Current CREBAIN documentation includes a managed real-NEST qualification workflow. It is not NCP qualification, and this review did not run it. “No real NEST work exists” would be false.
Codec ambiguity	The inspected validator already rejects duplicate encoder outputs and duplicate decoder channel/component pairs. The remaining concerns are missingness substitution, sparse layout, unit consistency, and a small-span interpolation error.
Idempotency eviction	Early eviction and unavailable results do not by themselves prove duplicate execution. Retry and state-version protections exist. The practical gap is a recoverable exact outcome under the declared retention contract.
Sensor receive time	Retaining receiver-local ingress time fixes time spent waiting after ingress. It does not, alone, prove total source/sample age before ingress.
“64 MiB memory use”	This is a maximum queued-payload capacity calculation for 64 maximum-size frames, not a measured resident-memory observation. Aggregate process limits require separate accounting.
“64 KiB allocation”	The writer reserves initial capacity up to 64 KiB. Reservation does not prove that all bytes are written, zeroed, committed, or resident per call.
Direct baseline	A direct baseline is an isolated test oracle only. It is not an allowed alternative runtime integration path in the delivered NCP-only system.

Earlier topic	Corrected or narrowed conclusion
Architecture originality	Prepared contexts, body-originated freshness, and mutually exclusive direct/gated authority already appear in NCP's proposed architecture. This report credits that work and tightens executable requirements; it does not claim those ideas were first invented here.
Verification language	Source inspection, proof under assumptions, abstract-model checks, native integration tests, and performance qualification are different evidence levels. None substitutes for the others.

Sources for the corrected factual boundaries: NEST lifecycle documentation, CREBAIN managed-runtime documentation, codec and idempotency source, bounded serialization, and NCP's proposed architecture. S04, S05, S07, S11, S13, S22

No measured synthetic detector rates from the earlier portfolio-wide review are carried into this report. They were not needed for this NCP handoff and were not independently re-established here.

3. Current consumer fit

3.1 CREBAIN

The managed simulator describes one to three declared entities, a 20 ms tick, a six-component position/velocity observation, and a three-component acceleration action. It uses Host API 2.0 with private length-prefixed JSON IPC, not NCP. Its standard profile has exact roster and time checks, explicit availability, and simulator-only authority. The older NCP bridge remains a separate historical integration surface. S13, S18

What must survive migration: the interpreted meaning of those vectors, action phase, per-entity availability, failure classifications, exact step plan, and local application receipt. Replacing the transport must not silently change the controller experiment.

What should not become universal NCP policy: the fixture's particular roster size, specific injected-fault schedule, initial positions, or controller gains. Those belong in an installed application profile.

A valid migration moves the externally reachable service boundary to NCP. It need not rewrite a correct local body kernel. It must not retain an Engram-to-CREBAIN private Host API channel as an undocumented fallback.

3.2 Prisoma

Prisoma's historical NCP observer is passive, wire-0.8-bound, and not qualified against a compatible live producer. Its documentation distinguishes exact source joins, absent language data, fixture evidence, and missing whole-tick detection. Separately, its managed observer contains explicit closed-loop timebase and causal step binding. S14, S15

The target is **general experiment capture first**, conditional scientific analysis second. An experiment with no language variable must remain a valid generic capture; it must not fabricate a language axis merely to satisfy a particular partial-information-decomposition analysis. Analysis eligibility, capture completeness, and transport conformance are distinct outcomes.

Preserve the strong receipt semantics of the managed observer while replacing its cross-project runtime path with NCP. Prisoma should own its evidence store; Engram sends capture data or retrieval requests through NCP rather than writing directly into Prisoma's database or filesystem.

3.3 Galadriel

The NCP architecture assigns Galadriel a read-only observation role and an isolated advisory-extension publishing role; it does not make Galadriel core authority or simulation truth. S11

The proposed integration must provide sufficient typed evidence for whichever monitor is actually installed. Merely delivering the same six kinematic values used by the controller may be insufficient for a statistical consistency monitor. Covariance, residual definitions, track identity, timestamps, and missingness may be required by the chosen detector profile. These are **proposed preparation requirements**, not an assertion that every current Galadriel entry point already demands exactly that schema.

A monitor must be able to abstain. Missing evidence, stale evidence, uncalibrated statistics, detector alarm, and transport failure must not collapse into a binary “safe/unsafe” result. De-tailed Galadriel schema and implementation verification remains mandatory before its adapter is accepted.

3.4 Haldir

Haldir is an experimental mission-authorization reference monitor. Its public design admits typed intent against policy and trusted state, then constructs its own NCP command; it is not a transparent command forwarder. Its runtime integration remains historical, and its documentation explicitly withholds production claims pending deployment and bypass-closure evidence. S19, S20, S21

Haldir is not needed merely to move NEST outputs into a deterministic simulator. It becomes useful when the experiment studies policy-constrained action, role separation, authorization failure, or deployment-like command gating.

A gated run must have an explicit Haldir requirement at preparation. Haldir cannot disappear on timeout and cause Engram to start publishing direct commands. An ungated run must declare Haldir absent rather than suggest that an unused gate somehow protects the path.

3.5 NCP’s role

NCP should own the shared contract, validators, transport bindings, conformance corpus, and narrowly scoped resource and identity rules. Project-specific simulation policy, statistical interpretation, and authorization policy remain with their owners.

The current low-overhead document is proposed architecture, not proof that these paths execute. Its direction is substantially compatible with the requirements here. Prefer implementing and testing that direction over maintaining a rival private dialect. S01, S11

4. Source findings and required corrections

Priority meanings: P0 affects experiment correctness, state integrity, or an enabled authority boundary. P1 affects required resource behavior, usability, or latency evidence. P2 concerns additional distribution or interoperability scope. A P0 finding is not automatically a security exploit.

4.1 P0: loss accounting occurs after intentional coalescing

In the inspected control-loop path, tick consumes the latest sensor and then calls `LinkMonitor::on_seq`. The monitor therefore sees consumed positions rather than necessarily every admitted ingress position. Its default reference and threshold are 0.05 and 5. S02, S03

Counterexample: all sensor positions 1 through 9 arrive, but a slower controller consumes 1, 5, and 9. Treating each three-position skip as loss produces successive accumulator values 2.8 and 5.6, crossing threshold 5. This is a consumption schedule, not evidence that three packets were lost on each interval. Section 9 derives the arithmetic; Appendix C checks it.

Correction: perform authenticated/session-bound stream accounting at ingress, before a latest-value replacement. Maintain separate counters for ingress gaps, rejected frames, duplicates, late reconciliation within its declared window, local coalescing, and consumed age. An ingress gap still does not prove where loss happened, much less malicious interference.

Acceptance: test 06 must deliver every frame and show zero ingress loss despite decimation. Do not disable real gap detection merely to make this case pass.

4.2 P0: consumption time is not original receipt time

The inspected path updates `last_sensor_t` with the tick's now when a sensor position advances. The latest-slot interface does not preserve an ingress timestamp in that returned sensor envelope. S02

An otherwise valid frame can wait during a controller stall and appear newly received when finally consumed. Carry immutable receiver-clock receipt metadata across the local boundary. Replays and unchanged positions must never refresh it.

That correction bounds only post-ingress age. For source-to-effect freshness, use the source-bound grant in Section 7. Never subtract unrelated monotonic clocks or renew a source budget at every relay.

4.3 P0: codec output can hide absence or ambiguous layout

The inspected codec validates finite ranges and rejects certain duplicate mappings, but missing inputs/readouts use midpoints, sparse components are zero-filled, and a channel's unit follows the last processed mapping. Its checked paths do not make an arithmetic midpoint universally neutral. S04

For a hypothetical command range $[0, 10]$, absent readout becoming 5 is a counterexample to universal neutrality. A fallback may be legitimate, but must have a declared disposition and must not masquerade as observed neural output.

Prepare an exact layout with availability semantics, consistent per-component units, and a declared missing-input policy. Reject unintentional holes and contradictions. Keep available zero, unavailable value, available empty spike window, and missing recording evidence distinct. A fixed-width absent slice can contain bounded inert bytes, but consumers must not interpret those bytes as measurements.

4.4 P0 for admitted scales: small ranges collapse in interpolation

`validate_range` allows strictly positive finite spans, while `lerp` treats spans smaller than machine epsilon as zero. The range $[0, 2^{-60}]$ is therefore admissible yet maps its upper endpoint to the output minimum in that helper. S04

This is a numerical consistency error, not a claim about current default UAV ranges. Machine epsilon near 1 is not a universal physical-unit tolerance. For finite validated nonzero spans,

compute the normalized fraction under an explicit overflow/underflow policy; alternatively reject unsupported scales during preparation. Add endpoint and unit-rescaling tests. The executable example returns 0 for the source pattern versus the mathematically required 200 for an output range [0,200].

4.5 P0/P1: idempotency protects mutation better than outcome recovery

The inspected store retains pending records during expiry cleanup; capacity reclamation can evict non-pending records before their nominal expiry. begin checks request timing and live authority before finding a prior record. Its terminal state stores a responder receipt, not an arbitrary full result payload. It also has explicit retry/state protections and an outcome_unknown outcome. S05

Do not infer demonstrated duplicate execution from eviction alone. The important design questions are whether the exact result remains retrievable, whether a permanently pending operation has a containment path, and what retention was actually promised.

For the lockstep profile, reserve one pending operation and one exact result until acknowledged. Preserve a high-water mark after payload release. A duplicate cannot become a fresh mutation merely because its payload was evicted. Authorize result lookup independently of new execution, with no unauthorized disclosure.

If execution may have occurred and its outcome cannot be reconstructed, retire the coupled generation. If serialization or result-budget enforcement fails after mutation, do not label the operation “rejected before execution.”

4.6 P1: bounded CUSUM work changes permitted detector behavior

The loss monitor caps per-gap observation work at 256 misses, while its configuration accepts larger positive thresholds. Its retained missing-position set is also finite. S03

For 1,000 misses followed by one delivery with reference 0.05, the uncapped final accumulator is 949.95; the 256-miss calculation is 243.15. A threshold of 300 separates those decisions. The default threshold would still be crossed, so this is not a claim that every large gap is missed.

Use the exact constant-work recurrence in Section 9, reject unsupported numeric regimes, and decide whether the detector reports the final state or any crossing during the batch. These semantics differ. State the bounded late-reconciliation policy rather than implying exact lifetime loss reconstruction.

4.7 P1: thread-local origins are not a movable object’s clock

The default clock helper uses a thread-local Instant origin. A loop that is moved and subsequently invoked on another thread can therefore encounter a different elapsed-time origin. The tick path has a non-regression check, so the issue can manifest as spurious failure/hold rather than a silent acceptance bypass. S02

Use a stable per-owner origin or a properly defined process-wide origin. Store elapsed integer durations where practical. A two-origin counterexample is checked in Appendix C; an actual cross-thread Rust regression is still required. This finding is conditional on migration and is not evidence that the current deployment actually migrates the loop.

4.8 P1: strict horizon behavior needs one executable definition

NCP’s own limitation ledger records a normative/executable horizon discrepancy and a generic TypeScript validation mismatch. S10

For future setpoint positions strictly before expiry, use the formula in Section 9, including the maximum TTL clamp and the count cap. Test exact expiry, fractional cadence, extreme finite inputs, and unsupported ratios through every public validator. A helper passing in one language is not generic-path conformance.

4.9 P1: exact contract verification accepts missing identity

`ContractStatus::is_match` accepts `NotAdvertised`; `verify_contract(None)` succeeds. That is documented behavior, but a name implying exact agreement is misleading. The generic decoder also materializes a JSON value before typed conversion. S06

Separate compatible-version status from proven prepared-contract identity. A strict prepared session requires the exact selected descriptor or explicitly negotiated compatible descriptor; absence is not proof of identity. Do not use a short advisory hash as authentication or as a collision-resistant evidence commitment.

4.10 P0 for secure profiles: transport identity is not established by a payload claim

The reviewed ingress feasibility study says the evaluated direct Zenoh interface does not provide the verified remote principal needed by the proposed secure admission path; that path remains unavailable rather than silently succeeding. The current grant structure principally expresses plane-level permissions rather than all proposed per-operation restrictions. S09, S12

This is a claim about the reviewed integration, not a claim that no future Zenoh version or deployment can supply suitable authentication. The implementation agent must verify the exact selected upstream version.

Keep unavailable security profiles fail-closed. Local private NCP transport must be selected and scoped explicitly; it must not be a silent downgrade. Bind authenticated or locally established origin to role, session generation, route, operation, audience, and resource scope at preparation and admission.

4.11 What is worth retaining

Retain bounded parsing, duplicate-key rejection, exact numeric identity rules, explicit generations, restrictive malformed-command handling, and the distinction between local admission and downstream execution. These are useful foundations, not expendable bureaucracy. S02, S06, S07

The optimization target is repeated work and unclear meaning. It is not removal of the checks that make untrusted messages safe to interpret.

5. Low-overhead implementation: remove repetition, not guarantees

The inspected transport validates sensor bytes and later decodes them again in the control adapter. Command handling performs bounded serialization before subsequent publication serialization; publication can copy byte slices. Its observation worker calls synchronous user code while draining a bounded queue. These are source patterns, not measured performance costs. S08

The bounded writer reserves up to 64 KiB initially. Active plant-command validation calls profile validation again. Neither observation establishes the time saved by changing it. S07, S17

5.1 Prepared objects and typed ownership

Introduce a small validated object graph:

`PreparedRun`

- identity and installed semantic descriptor
- exact time plan and participant bindings
- immutable layouts and explicit availability rules
- resource reservations and selected authority mode

`Received<T>`

- validated typed value or immutable validated bytes

exact ingress identity and receiver-local receipt time
ownership of its resource reservation

These names are proposed design labels, not existing API declarations.

Decode once per trust boundary and keep the result. Pass immutable ownership rather than raw bytes that force the next layer to rediscover meaning. Serialize the final committed publication once after its final stream identity is assigned. A pre-execution size check must still guarantee that the eventual output is representable within its reservation.

Static checks include unit compatibility, bounds, layout shape, canonical descriptor identity, and installed policy structure. Dynamic checks include current generation, next expected step, actor scope, lease/currentness where applicable, original source grant, numeric payload validity, and result capacity.

Removing repeated profile hashing is safe only if the object is immutable or replaced atomically by a new generation. A shared mutable object with a stale “validated” flag is not a prepared contract.

5.2 Queue and memory policy

Bound aggregate retained bytes, not just item count. The illustrative capacity of 64 one-MiB payloads is 64 MiB before queue objects or transport storage. This calculation does not assert that normal small-frame operation consumes that amount.

For every queue, specify owner, item ceiling, byte ceiling, maximum age, overflow decision, release condition, and teardown behavior. Include transport buffers, result retention, extension attachment reservations, and fan-out. Sharing one immutable payload can reduce copies, but each additional retention reference still delays reclamation and must be accounted for.

A useful conservative reservation equation is

$$M_{reserved} = M_{fixed} + \sum_j B_j + \sum_s R_s + M_{scratch},$$

where B_j bounds each owned queue and R_s each retained result. Count shared allocations once in actual memory telemetry, but do not overcommit reservations on the assumption that sharing will always occur.

A full queue rejects work before state mutation or applies the selected coalescing/drop policy. It must not accept a step, mutate NEST, and only then discover that the required result cannot be retained.

5.3 Observer isolation

An asynchronous function name does not isolate synchronous user work. Use a bounded dedicated worker or a separate process when callbacks can block indefinitely. Do not spawn unlimited replacement tasks for stalled consumers.

Recording, statistical analysis, visualization, compression, and remote upload stay outside the fast path unless the profile explicitly makes their result a causal dependency. For lossless lockstep capture, reserve space before advancing; pausing can occur at a defined boundary. For real-time best effort, record loss and do not claim complete evidence.

Use immutable observation projections with no commander credentials. A GUI, logger, or embedded presentation panel cannot become a hidden source of controller input or authority.

5.4 Transport selection

Start with a small strict NCP local binding over a private process channel or properly permissioned local socket. Its framing, operations, errors, resource admission, and conformance corpus belong to NCP. The fact that the underlying channel is a pipe does not make it non-NCP; the fact that a payload is labeled NCP does not make an arbitrary legacy API conforming.

Preserve a network binding where it is useful. Add compact fixed-layout frames only after profiling shows that small-message encoding/copying is material. A binary encoding must retain version identity, bounded lengths, byte order, finite-number policy, exact integers, and availability padding rules.

Do not mandate shared memory, zero-copy, lock-free queues, busy polling, real-time scheduling, or per-frame signatures by default. Each has assumptions and operational costs. Optimize the measured bottleneck for the selected profile.

6. The NCP-only ecosystem boundary

6.1 Allowed topology

Every arrow below is an NCP project-to-project interaction. Private execution inside one owning service is not an extra ecosystem API.

DIRECT SIMULATION MODE

```
Engram [orchestrator, NEST owner, sole commander]
  <---- NCP ----> CREBAIN [body state, final application]
  <---- NCP ----> Prisoma [capture, analysis, retrieval]
  <---- NCP ----> Galadriel [observation, advisory output]
```

GATED MODE

```
Engram -- NCP typed intent --> Haldir -- NCP command --> CREBAIN
CREBAIN -- NCP observation/disposition --> authorized consumers
Engram -- NCP neural evidence --> Prisoma and Galadriel
Galadriel -- NCP advisory --> Haldir and authorized Engram roles
Haldir -- NCP decision evidence --> Engram and Prisoma
```

All required evidence reaches Prisoma through NCP.
Engram has no direct plant-command capability in gated mode.

The drawings express role relationships, not a required centralized routing service. Observation fan-out can be direct to subscribed NCP peers. Haldir does not need to forward all telemetry. Credentials and routes should be granted to roles, not indiscriminately to an entire repository or desktop process.

The direct/gated separation agrees with NCP's proposed ecosystem direction. Its integrated Haldir proposal also separates extension ingress, policy state, and commander responsibilities. This report preserves the separation but requires measured, threat-model-driven process placement rather than assuming that more processes are automatically safer or faster. S11

6.2 Explicit prohibitions

The delivered Engram runtime must not integrate with these projects through direct engine imports, a parallel Host API, custom HTTP/WebSocket endpoints, ad hoc pipes, shared mutable storage, file polling, or manually interpreted raw transport taps. It must not smuggle the old interface through a generic "execute" extension.

A project-local worker is allowed only within the owner's declared trust boundary. For example, Engram may privately supervise a NEST worker that has no CREBAIN, Haldir, Galadriel, or Prisoma credentials. CREBAIN may keep its simulation kernel as an internal library. These are not permission to host another project's externally controlled runtime under an Engram-shaped name.

Cross-repository editing, compilation, downloaded build dependencies, and isolated unit-test baselines are development activities, not production integration paths. Their presence must not leave a runtime bypass enabled in a qualified package.

6.3 Enforceable connection policy

At preparation, install a role allowlist binding peer identity, allowed operations, exact session kind, route or endpoint, and direction of data flow. Refuse unknown peers and implicit fallback. Give observer roles no mutation capability. Give the gate-intent publisher no final-command capability.

Enforce this in layers: package/dependency checks for forbidden adapters, endpoint configuration, narrowly issued credentials, sandbox egress and inherited-handle policy, and runtime connection tracing. Static searches alone miss dynamic imports and runtime routing; packet

capture alone misses local shared-state bypasses. Test both the intended route and deliberate bypass attempts.

The agent should be unable to acquire command authority by reading a GUI projection, calling a helper process with broad inherited credentials, or writing a sidecar file. An evidence dashboard is not a privileged integration surface.

6.4 Resource and code authority during simulation creation

Engram may generate and validate a simulation specification and create its local NEST network. Externally supplied model configuration is data under a bounded schema, not unrestricted Python execution requested by a peer.

Prepare separately budgets model size, connection count, initialization memory, execution duration, and recording volume. A cheap per-step frame limit does not prevent a configuration from requesting an enormous simulation. Cancellation during construction must have an explicit cleanup and worker-retirement path.

7. Execution contract: exact steps, original freshness, known outcomes

All operation names and data structures in this section are proposed semantic requirements. Reuse compatible existing NCP operations where possible; introduce explicit versioned extensions where necessary. Do not create a second private contract.

7.1 Prepare

A prepared descriptor binds the run identity and generation, each participant's role, exact semantic contract identity, participant generations, roster and layout, neural model/build identity, seeds and reproducibility policy, timebase, number of steps, sampling/application phase, permitted intervention points, authority mode, observer mode, and resource reservations.

It also declares which values are *data*, which are *availability*, which are *proposals*, and which are *evidence of application*. A descriptor hash records identity; its presence does not establish that the producer implemented the descriptor honestly.

prepare must not itself create final command authority from a caller's self-asserted role. The receiver binds authority to the established principal and selected session. A compact local capability can be opaque and receiver-owned; it need not be an elaborate transferable token.

7.2 Step

For a simple sample-neural-run-apply schedule:

$$y_k = H(x_k), \quad (z_{k+1}, u_k) = N(z_k, y_k), \quad x_{k+1} = P(x_k, u_k).$$

The state includes all relevant neural, plant, controller, random-generator, and online-monitor state. The input y_k includes declared availability, not just numeric values. P includes application-time constraints and may produce a bounded or fallback action distinct from u_k .

A step request identifies the prepared generation, exact step, exact input predecessor, operation identity, and bounded payload. The result identifies the exact request, resulting state position, applied or proposed action as appropriate, and terminal outcome.

One dependent step is outstanding. Do not batch future steps that depend on as-yet unknown plant observations. Batching independent sessions, transport writes, or evidence records is different and may be useful.

Do not require full-state hashing on the hot path solely for appearances. Bind the needed identities and exact result; compute larger evidence commitments on a reserved, ordered capture path when that satisfies the selected completeness policy. A hash chain cannot substitute for missing causal fields.

7.3 Local outcome state machine

Ready

```
-> RejectedBeforeExecution -> Ready
-> Executing
    -> CommittedWithRetainedResult
        -> ResultAcknowledged -> Ready
    -> Indeterminate -> Retired
```

Ready -> Finished

A known failure before mutation is not indeterminate. A failure after mutation is not necessarily committed unless the backend's commit boundary and outcome are actually known. The API must expose these distinctions instead of using one generic timeout exception.

A matching retry retrieves the same retained result. Same operation identity with different semantic input is a conflict. Result lookup does not rerun the model. Payload acknowledgement releases retained bytes but not the high-water mark that prevents stale execution.

Use an explicit acknowledgement or a precisely defined next-step acknowledgement. The next request can acknowledge the prior result only when the requester possesses that exact result identity; merely requesting a higher index is insufficient.

No unbounded pending cache is necessary for one-in-flight lockstep. General concurrent life-cycle operations can use a different bounded store, with their own contracts.

7.4 Coupled commit is not an atomic transaction

NEST and CREBAIN are separate state owners. Without a qualified rollback/checkpoint mechanism, the orchestrator cannot make their advances atomic merely by wrapping them in a method called step.

Distinguish `NeuralCommitted(k)`, `BodyCommitted(k)`, and `PairClosed(k)`. The last means the required exact outcomes are known and joined—not that a distributed database transaction occurred.

If the neural advance completes but body application is uncertain, stop advancing the coupled run. Retire the run generation and participating mutable state that would otherwise be reused inconsistently. Preserve the maximal closed prefix and the incomplete suffix as different evidence classes.

A recovered result can resolve a temporary uncertainty only under a specified recovery protocol while no dependent mutation has proceeded. In the initial lean profile, prefer bounded result recovery followed by retirement on unresolved uncertainty. Do not implement opaque speculative recovery.

Independent sessions can continue. A failure in one run does not justify a global process lock or the destruction of unrelated experiments.

7.5 Source-bound freshness without synchronized clocks

For a real-time sampled profile, CREBAIN can issue an opaque source grant with each observation. Before publishing, the body owner records the source identity and a deadline on its own monotonic clock. The returning command carries that exact reference; relays do not restart its lifetime.

At final application the body checks the reference against its retained record and compares current body time with the original body deadline. Both times belong to one clock. This includes outbound transport, computation, gate traversal, queuing, and return transport without an extra freshness round trip.

Use a bounded retained-grant window. If the source record has been evicted, reject or use the declared fallback; never substitute “latest.” Authentication and exact generation binding are still required. A guessed source identifier is not authority.

This bounds age from the body's recorded event. If a camera captured the observation earlier,

its acquisition age needs a trusted mapping or conservative bound as well. The mechanism does not magically prove external sensor time.

For deterministic lockstep, exact predecessor and step checks govern scientific validity. A wall-clock timeout remains a liveness/containment policy, not permission to relabel an old logical input as current.

7.6 Cancellation, restart, and finish

Cancellation is a request to stop further work, not evidence that a synchronous backend rolled back. A non-returning NEST worker may require process termination. An uncertain old generation remains retired after restart.

A new run gets new generation identities, fresh neural and body state or an explicitly qualified checkpoint, and a fresh authority context. Do not reuse an old latest-value slot or observer join state.

Finish states planned and completed step counts, the last closed pair, any incomplete suffix, capture completeness, outstanding dispositions, and the reason for termination. A partial run can be useful evidence, but must not be represented as successful completion of the exact full plan.

7.7 Final-effect currentness and revocation ordering

Application-time checking needs a precise event order. The body owner should serialize receipt admission, source-grant validation, authority changes, and effect invocation under one bounded ownership model. No user callback, network call, or long simulator operation may run under a realm-wide authorization lock.

Define revocation effectiveness as an event accepted by the designated authority/body owner, with an explicit ordering relative to applications. A command ordered after effective revocation cannot use the old term. A command already irreversibly applied before that event cannot be retroactively undone by a later message. Network delivery of a revocation request is not the same as its acceptance at the final executor.

If authority can change concurrently outside that owner, use a versioned currentness mechanism with a final recheck that actually closes the check/effect race under the implementation's concurrency model. A stale pointer to a "verified" policy snapshot is not sufficient. The initial local simulation profile can avoid unnecessary concurrency by keeping all mutation in its single owner.

Prepared extension receivers must not increase a capability's scope. An advisory or intent admission receipt can bind evidence and narrow a decision; it cannot be exchanged for a broader role token. Emergency messages still require the correct authenticated/local origin and exact live generation, even when they do not require an ordinary active-command lease.

8. Consumer-specific implementation requirements

8.1 Engram and NEST

NEST documents persistent run control through Prepare, Run, and Cleanup, and a RunManager; Simulate performs preparation/cleanup and is not equivalent to ResetKernel. ResetKernel destroys network state. The documentation restricts which changes are meaningful during prepared execution. `local_spike_counter` is reset per Run. S22

Implementation: build the network once per run generation, preserve state across steps, and qualify the exact update path. Create a support matrix naming NEST version/build, model/device, allowed parameter mutation, calibration boundary, effective interval, and observed response. Unsupported live changes fail during preparation, not after silently doing nothing.

Record the neural resolution, stimulation schedule, event-readout convention, and simulation-advance duration. Every requested advance must be representable by the selected backend. A rational common timebase does not make an arbitrary real-valued duration executable by

NEST.

Separate reset-per-run counters from cumulative counters. For event records, determine the exact inclusion convention and precise-time representation with boundary tests. NEST's spike recorder reports spike creation times and documents its recording interval behavior; do not infer full window semantics from a generic timestamp name. S23

Avoid repeatedly copying an ever-growing historical event array to obtain the latest window. If work at step k is proportional to k , cumulative extraction work over n steps grows as $n(n+1)/2$. Use a bounded current-window path or a suitable declared counter; reserve full event capture separately.

Population-rate readout must bind population membership, available event count, window duration, and effective population size. An empty observed window is zero events; a failed readout is unavailable. Rate-estimation noise belongs in the controller design, not in a misleading protocol latency claim.

Record thread/MPI placement and runtime contention. Reproducibility must be declared as bitwise, tolerance-based, or statistical for the selected build and execution configuration. Protocol identities do not make arbitrary parallel simulations bitwise reproducible.

8.2 CREBAIN

The target adapter must preserve position, velocity, and acceleration as different quantities with explicit coordinate frames. Reuse the current deterministic body update semantics as a baseline, including availability and local commit behavior, without making its fixture-specific choices universal. S13

The receiver must distinguish a controller proposal, a bounded proposal, a fallback disposition, and the action actually applied. Return the exact source reference and resulting state position. Current authority and source validity are checked immediately before effect, not only when a message is placed in a queue.

Zero acceleration is not a universal physical stop. In this simulator it may be a declared hold action; velocity can remain nonzero. The physical-plant profile needs its own actuator semantics, watchdog, and independent deployment evidence.

Use a test-only direct oracle to compare numerical trajectories with the NCP adapter. The distributed runtime must not be able to switch to that oracle when the NCP connection fails.

8.3 Prisoma

Define a project-neutral causal capture record that can bind preparation, source observation, neural request/result, controller proposal, gate decision when selected, body disposition, resulting observation, and terminal run status. Preserve each event's actual producer role.

Completeness cannot be inferred only by joining the records that happened to arrive. Retain a run plan and terminal/high-water evidence so omission of an entire step is detectable. If the terminal record is missing, mark completeness unresolved, not successful.

Out-of-order delivery requires bounded joins by exact identity, not nearest timestamp. Exact duplicates can be idempotent; changed payload under the same committed identity is a conflict. Large attachments require separately budgeted ownership and immutable identities. They must be transferred through a declared NCP mechanism, not via arbitrary remote URLs or a shared directory that bypasses the boundary.

An evidence hash attests to byte identity under its construction. It does not prove that NEST really ran, that a remote sensor was truthful, or that a compromised producer did not fabricate a result. Scientific claims require the relevant independent evidence.

Generic capture must be useful even when a specific PID analysis is ineligible. Missing variables remain absent. Do not require a per-tick PID calculation, synthesize a language embedding from a hash, or put an offline estimator on the control path to make a dashboard look complete.

8.4 Galadriel

Prepare a named monitor profile that specifies accepted evidence, component/unit conventions, required covariance or uncertainty metadata, track/run identity, missing-data behavior, calibration identity, output vocabulary, and validity interval. The actual Galadriel implementation must be checked against this profile before claiming compatibility.

For an innovation-based example, a residual is $r = y - \hat{y}$ and a positive-definite innovation covariance is S . The normalized statistic is

$$q = r^T S^{-1} r.$$

An implementation should solve using a suitable factorization rather than explicitly computing an inverse. If r is zero-mean Gaussian with covariance $S = LL^T$, then $v = L^{-1}r$ has identity covariance and $q = v^T v$ is a sum of squared independent standard-normal components. This derives the chi-square reference law under those assumptions; calling a matrix “covariance” does not establish them. A singular or invalid covariance requires an explicit unsupported/abstention path, not undisclosed numerical repair. Temporal correlation, parameter fitting, multiple testing, and missing observations affect calibration. A large statistic establishes inconsistency relative to a model, not malicious intent.

These are mathematical design requirements, not claims that the current detector implements every listed condition. The first cross-repo Galadriel task is to inventory its real evidence schema, accepted models, and documented limits.

Separate two modes. In **monitoring mode**, assessments are asynchronous and cannot affect the experiment’s state transition. In **online-guard mode**, the required assessment for step k is a declared dependency with a deadline and explicit stale/abstention handling. A nondeterministically selected “latest assessment” is not acceptable in a deterministic run.

An advisory may preserve or remove existing permission according to policy. It cannot create command authority. A transport failure is not automatically a detector alarm, and a detector’s abstention is not automatically a nominal result.

8.5 Haldir

The role is policy-constrained admission of intent, not neural computation, sensor truth, or evidence storage. Existing public Haldir material separates publication from downstream application and does not establish a production deployment. Its declared-live limitations include the lack of adequate application/freshness evidence for continued live operation after publication. S19, S21

In gated simulation mode, Engram publishes semantic intent through a registered NCP extension. Haldir independently checks actor, exact run context, policy, input currentness, and relevant advisory conditions. It then constructs a new command under its own sole body-issued authority. It does not re-sign or forward Engram’s command bytes as though authority had transferred with them.

CREBAIN still makes the final admission and effect decision. Haldir must reconcile the body disposition rather than infer success from a successful publish call. Source grants and application receipts address necessary protocol gaps, but do not by themselves solve credential custody, bypass closure, or physical deployment safety.

Keep the intent receiver, advisory receiver, policy owner, and commander capabilities distinct. Process isolation can enforce meaningful trust separation, particularly around untrusted extension parsing. The simulation preview can choose a smaller explicitly weaker local profile, but must not represent it as the fully isolated deployment profile.

On missing or indeterminate Haldir output, use the declared stop/hold/termination policy. Do not fail open to direct Engram command. Changing authority mode requires a new prepared authority term or run generation with independently admitted scope.

9. Mathematical contracts and executable checks

The proofs below are under stated mathematical assumptions. Appendix C checks finite instances and counterexamples. It does not constitute formal verification of the repositories, machine arithmetic in every regime, or a physical controller.

9.1 Strict horizon count

Let $L = \min(TTL, TTL_{max}) > 0$, cadence $d > 0$, and integer count cap $C \geq 0$. Future setpoint j executes at offset jd for integer $j \geq 1$. Expiry is strict: $jd < L$.

The largest allowed integer is therefore

$$N = \min \left(C, \max \left(0, \left\lceil \frac{L}{d} \right\rceil - 1 \right) \right).$$

Proof. For positive $x = L/d$, integers satisfying $j < x$ are exactly $1, \dots, \lceil x \rceil - 1$. Apply the nonnegative and count caps. For integer time ticks, use the overflow-aware equivalent $\lfloor (L - 1)/d \rfloor$ before the count cap. Zero/invalid lifetimes are rejected or handled explicitly before this positive-input formula.

At $L = 40$ ms and $d = 20$ ms, only the future position at 20 ms is valid; the position at 40 ms is expired. A $\text{floor}(L/d)$ rule would include one too many at exact division. Exact integer/rational arithmetic also avoids overflow in a floating-point TTL/d ratio.

Check: enumeration of executable positions over a finite rational grid, plus integer-boundary cases. This validates the formula, not current TypeScript or Rust parity.

9.2 Constant-work CUSUM batching

For nonnegative accumulator s , reference $0 \leq r < 1$, and $m \geq 0$ missing events followed by one delivered event, define a loss update as $s \leftarrow \max(0, s + 1 - r)$ and a delivery update as $s \leftarrow \max(0, s - r)$.

Because every loss increment is nonnegative, the exact final state is

$$s' = \max(0, s + m(1 - r) - r).$$

This replaces m updates with constant arithmetic work. It does not make finite machine arithmetic exact; choose checked fixed-point/rational behavior or a justified floating-point error policy and handle huge sequence gaps explicitly.

For consumed positions 1, 5, 9, each observed gap has $m = 3$. With $r = 0.05$ and initial $s = 0$, the states are 2.8 then 5.6. With $m = 1000$, the state is 949.95; truncating to $m = 256$ gives 243.15.

Important distinction: whether any intermediate event crossed a threshold is not identical to whether the final state exceeds it. A delivery can reduce an accumulator below threshold. If transient crossing latches a condition, compute the peak as well and define exactly which histories matter. Do not change detector semantics accidentally during optimization.

9.3 Exact common timebase

For positive rational durations $d_i = p_i/q_i$, let Q be the least common multiple of denominators and $a_i = p_i(Q/q_i)$. A common quantum is

$$\delta = \frac{\gcd(a_1, \dots, a_n)}{Q}.$$

Then every d_i/δ is an integer. Store logical time as integer tick position and compute $t_k = k\Delta$ without repeated floating-point accumulation. Check representability and overflow before preparation; denominator growth must be bounded.

For a 0.1 ms neural resolution and 20 ms control interval, 200 neural ticks make one control interval. One million intervals correspond exactly to 20,000 seconds under that declared rational model.

This construction does not prove that a backend accepts the chosen quantum or that a particular live-update mechanism acts at its boundary. Those are integration tests.

9.4 One hidden step of delay changes stability

Consider the scalar integrator with feedback

$$x_{k+1} = x_k + \Delta u_k, \quad u_k = -Kx_k,$$

and define $a = K\Delta$. Without added delay, $x_{k+1} = (1 - a)x_k$, which is asymptotically stable exactly when $0 < a < 2$.

If an integration error applies the previous observation instead,

$$x_{k+1} = x_k - ax_{k-1}.$$

Its characteristic polynomial is $z^2 - z + a$. For $0 < a \leq 1/4$, the real roots lie inside the unit circle. For $1/4 < a < 1$, they are complex conjugates with product a and modulus $\sqrt{a} < 1$. At $a = 0$ or $a = 1$ a root lies on the unit circle; for $a < 0$ a real root exceeds 1, and for $a > 1$ the product of root magnitudes exceeds 1. Thus stability is exactly $0 < a < 1$.

At $a = 1.5$, the undelayed pole is -0.5 , but delayed poles have modulus $\sqrt{1.5} > 1$. A nominally instantaneous transport with the wrong application phase can therefore be worse than a slower correctly phased one.

This is an illustrative controller, not a proof that CREBAIN's nonlinear bounded dynamics obey this model. It establishes why phase equivalence deserves a P0 test.

9.5 Deterministic direct/NCP equivalence

Assume the full state transition F is deterministic for a fixed initial state, fixed random-generator state, declared inputs, and schedule. Assume an adapter preserves every semantically relevant input and invokes exactly the same transition once per step.

If direct and adapted states agree at k , their next inputs and transition agree, so their states agree at $k + 1$. Since initial states agree, induction gives equality for all completed steps.

The assumptions expose the test obligations: no added delay, no missed availability, no changed saturation, no duplicated advance, no different random stream, and no hidden wall-clock input. Hash equality is useful evidence of selected byte identity, but not a proof that all relevant hidden state was included.

For approximate implementations, if $\|F(x) - F(y)\| \leq L\|x - y\|$ and each step adds at most ϵ mismatch, then $e_{k+1} \leq Le_k + \epsilon$. With $e_0 = 0$, induction yields $e_k \leq \epsilon \sum_{i=0}^{k-1} L^i$. This can grow rapidly when $L > 1$; an arbitrary per-step tolerance is not automatically a meaningful long-run trajectory guarantee.

9.6 Freshness on one receiver-owned clock

Let the body record a source event at b_0 and deadline $b_0 + B$, with $B > 0$. It accepts the returning command only when its current clock $b_a < b_0 + B$. Then elapsed time on that clock satisfies $b_a - b_0 < B$ by subtraction. No remote clock offset appears.

With 15 ms outbound delay and 15 ms return delay, a 20 ms source budget is expired at return. Restarting a 20 ms lifetime at the remote receiver after the outbound leg would incorrectly accept the same return. The grant avoids that reset.

Assumptions matter: the same body clock is monotonic over the interval, the source identity is exact, the original deadline is retained, and intermediate processing cannot forge a fresh

source grant. Relating clock duration to physical elapsed time requires an oscillator/clock-error bound; no hard physical-time guarantee is claimed without one.

9.7 Closed-prefix consistency and at-most-once execution

Maintain counts n for known neural commits, p for known body commits, and c for closed pairs. For one-in-flight coupling, require

$$c \leq p \leq n, \quad n - c \leq 1.$$

Transitions preserve the inequalities: neural commit is allowed only from ready; body commit only after its neural commit; close only after both. Retirement permits no further same-generation transitions. This is a state-machine property, not distributed atomicity.

At-most-once mutation additionally needs a unique operation identity bound to semantic input, serialized ownership, retained/high-water state, and no transition from replay/unknown to fresh execution in the same generation. After a crash, this property may require retiring the generation rather than trying to reconstruct forgotten state.

The finite models in Appendix C explore bounded histories of these proposed rules. They do not model the real message parser, threads, process crashes, persistence implementation, or all possible histories.

9.8 Recording cannot evade conservation of backlog

Suppose a producer continues at rate p , a recorder drains at rate $c < p$, and retained buffer capacity is B . Backlog increases at rate $p - c$ and fills in at most $B/(p - c)$ after an empty start under the constant-rate model.

Therefore bounded memory, indefinite operation, lossless delivery, and a nonblocking producer cannot all be guaranteed against an arbitrarily slow recorder. Choose a profile-specific consequence: pause, reject/terminate, or drop and mark incompleteness.

A conditional fluid bound is also useful. If arrivals satisfy $\alpha(t) = b + rt$ and FIFO service provides $\beta(t) = C(t - T)^+$ with $b, r, T \geq 0$ and $C > 0$, $C \geq r$, then backlog is at most $b + rT$ and delay at most $T + b/C$. The latter follows because $C(t + b/C) = Ct + b \geq rt + b$. These bounds require the declared arrival and service envelopes; ordinary operating-system scheduling does not automatically supply them. Packetization and already-in-service storage need separate treatment.

9.9 Rate-code noise versus feedback window

Under an explicit independent Poisson assumption, $N \geq 1$ neurons each firing at rate $\lambda > 0$ over window $\Delta > 0$ produce aggregate count $K \sim \text{Poisson}(N\lambda\Delta)$. For $\hat{\lambda} = K/(N\Delta)$,

$$\text{Var}(\hat{\lambda}) = \frac{\lambda}{N\Delta}, \quad CV = \frac{1}{\sqrt{N\lambda\Delta}}.$$

At 20 Hz and a 20 ms window, the illustrative coefficient of variation is about 15.81% for 100 independent neurons and 55.90% for eight. These are model calculations, not measured NEST results. Correlation, refractory behavior, input structure, and readout filtering change the model.

Shorter windows reduce observation delay but increase noise under these assumptions. Extending a window changes the controller's information and phase; it is not a free protocol optimization. A very fast transport cannot remove that estimation tradeoff.

For equal marginal variance and pairwise correlation ρ , aggregate variance is multiplied by $1 + (N - 1)\rho$ relative to independence, when such a covariance model is valid. Thus increasing neuron count does not guarantee the independent-neuron reduction in noise.

9.10 Advisory authority is monotone

Let A be the set of actions permitted by independently established policy and authority. Let G be an advisory-derived restriction. Compose as $A' = A \cap G$. Then $A' \subseteq A$ by set intersection.

This simple property is the desired direction for Galadriel-to-Haldir influence. A monitor may remove permission or leave it unchanged; it must not mint a lease or expand A . The policy still decides how absence, staleness, and abstention affect G . This is monotonicity relative to independently granted A , not a claim that every restriction is irreversible over time. Restoration and latch reset require their own explicitly authorized policy. The finite powerset check establishes the set identity, not correctness of the real authorization implementation.

10. Performance claims and measurement design

10.1 Define the complete quantity

Measure source observation to actual body application, not merely controller computation:

$$L_{loop} = L_{ingress} + L_{queue} + L_{map} + L_{neural} + L_{readout} + L_{control} + L_{gate} + L_{return} + L_{apply}.$$

Set $L_{gate} = 0$ only in a profile without that dependency. Include observation generation when the question is sensor-to-effect rather than publication-to-effect. Instrumentation must state where every timestamp is taken.

The inspected `loop_latency_ms` timer ends before command submission and therefore does not measure this full chain. Existing performance documentation also distinguishes microbenchmarks from complete qualification. S02, S16

10.2 Scale the problem before optimizing

Three entities with six observation doubles and three action doubles contain $3(6 + 3)8 = 216$ numeric bytes per step, or 10,800 bytes per second at 50 Hz. This excludes envelopes, neural outputs, evidence, and fan-out. It is an illustrative payload calculation, not observed traffic.

For that small case, allocator behavior, scheduling, repeated parsing, backend invocation, and semantic phase errors are plausible first targets. Large spike recordings or rich perception evidence can have completely different bandwidth needs and should not share an unbounded fast-path queue.

An initial local added-overhead goal of 1% of a 20 ms interval is 200 microseconds at a chosen tail percentile. This is a proposed target—not current performance, a forecast, or a hard real-time guarantee. Select hardware and load before adopting it as a release requirement.

10.3 Paired baseline methodology

Compare a direct test-only baseline and NCP-bound execution with identical model, controller, input schedule, seeds, CPU placement, and output interpretation. Randomize or alternate comparable runs to reduce time-dependent bias. Record instrumentation overhead.

Do not subtract unrelated p99 values and call the result p99 protocol overhead. Use stage measurements on the same execution or a justified paired design. Across hosts, use same-clock round-trip measurements unless clock synchronization error is measured and included in one-way uncertainty.

Report p50, p99, p99.9, maximum observed, sample count, warm-up policy, deadlines, rejected work, missed work, allocation count, copy volume, resident/retained memory, and throughput. Maximum observed is not a worst-case execution-time proof.

Drive the intended arrival schedule even during stalls, or explicitly count arrivals that the generator could not offer. A request generator that waits for every completion can hide overload by reducing offered load.

10.4 Qualification matrix

Run the selected workload with observer absent, normally active, deliberately slow, full, disconnected, and non-returning. Test multiple independent sessions and fairness. Separately test local transport, network transport, and gated mode rather than combining their numbers into one claim.

Record NEST build/model configuration, threads/MPI, machine and operating-system configuration, process placement, power policy where relevant, compiler options, binding versions, and exact package identities. Saturation or fail-closed behavior is part of the result, not an outlier to delete.

Do not pursue a claimed “zero overhead” result. The useful objective is a small, measured, bounded contribution that preserves the experiment’s semantics.

11. What to simplify—and what not to oversell

The problematic “slop” is not that the repositories contain no implementation. It is the risk that terminology, architecture breadth, or assurance artifacts imply more than the executable integration establishes.

Simplify names. Distinguish version compatibility, exact prepared-contract agreement, local slot admission, publication, receiver admission, and application. A method returning success should name which of those occurred.

Simplify contract ownership. Maintain one normative semantic definition plus an executable cross-language boundary corpus. Generated schemas, prose, and validators must not independently invent expiry, missingness, or identity semantics. A hash of a schema does not capture unstated application behavior.

Simplify the local profile. One prepared role context, one backend owner, one outstanding step, and one retained outcome are enough for the initial deterministic loop. Do not force a local simulator through every remote physical-deployment mechanism.

Do not simplify away authority. When Haldir or a physical profile is selected, role separation and final-effect checks are real requirements. A local fast path must not silently become a weaker path under the same security label.

Prefer executable evidence. A small failing regression, a reproducible native loop, and a measured resource envelope are stronger evidence than additional review counts, broad “verified” labels, or speculative architecture diagrams. Negative results and explicit limitations should remain visible.

Avoid fake universality. NCP need not solve arbitrary multi-simulator co-simulation, distributed rollback, event-time causality, physical safety certification, and every information-theoretic analysis in its first useful local profile. Supporting a narrow correct use case is compatible with a clean extension boundary.

The design advances worth pursuing are specific: source-bound freshness with no relay reset; closed-prefix recovery rather than false atomicity; typed missingness that survives codecs; constant-work loss updates with exact declared semantics; and NCP-only role capabilities that cannot be bypassed by a convenient legacy adapter. Several foundations already exist in proposed NCP work. The contribution here is to make their failure cases and acceptance obligations concrete.

12. Cross-repository implementation handoff

12.1 Gate 0: obtain actual source identities before changing code

The agent must identify the real Engram host checkout rather than modifying the README-only placeholder by assumption. For each participating repository, record origin URL, full commit ID, dirty status, submodule identities, lockfiles, toolchain versions, and source/package hashes relevant to the build.

The following commands are an inspection recipe, not commands executed during this audit:

```
git -C "$REPO" remote get-url origin
git -C "$REPO" rev-parse HEAD
git -C "$REPO" status --porcelain=v1
git -C "$REPO" submodule status --recursive
```

Reject an unrecorded dirty tree for qualification. Preserve the baseline before cross-repo edits. Confirm each source finding by its named symbol, write a minimal native regression, and update the claim ledger if current code differs. Do not mechanically apply an old finding to changed code.

12.2 Change set A: correctness in NCP core

Existing surfaces to inspect include `ncp-core/src/transport.rs`, `resilience.rs`, `codec.rs`, `idempotency.rs`, `messages.rs`, `plant.rs`, and `security.rs`.

Implement ingress accounting and original receipt metadata; stable clock ownership; explicit missingness and prepared layouts; small-span interpolation semantics; constant-work CUSUM; strict contract preparation; consistent horizon vectors; and a bounded exact-outcome contract.

Each change starts with a failing native regression and ends with public-path tests. Keep legacy behavior only where explicitly versioned and not selected by the new prepared profile. Do not silently change the interpretation of an immutable released wire version.

12.3 Change set B: neutral prepared execution contract

Add or revise NCP-owned definitions for the time plan, participant generations, prepared layout, exact step identity, source grant, operation outcome, application disposition, and terminal completeness. Reconcile with the existing proposed B01 design instead of creating parallel names and receipt families.

Keep the universal core small. Project-specific intent, consistency evidence, and monitor assessment belong in registered bounded extensions with exact independent role grants. Extension registration does not automatically grant authority or permission to fetch arbitrary attachments.

Define compatibility deliberately. An unpublished candidate can change incompatibly with a new identity and explicit migration. Never rewrite the meaning of an immutable historical tag or advertise wire-0.8 fixtures as native-1.0 qualification.

12.4 Change set C: local NCP binding and fast path

Inspect `ncp-zenoh/src/lib.rs` for retained typed results, duplicate serialization, copying, ob-server scheduling, and resource reservations. Add a project-neutral local binding only if the selected existing binding cannot meet the intended local boundary cleanly.

The local binding must execute the same public semantics and boundary corpus, not a reduced undocumented test protocol. Provide explicit local-origin establishment and process ownership. Do not mark it remote authenticated or physical-qualified merely because a private pipe works.

Counter instrumentation should expose serialization count, parse count, profile validation count, owned bytes, result retention, and queue age. Start with correctness and measurement; add compact encoding only with demonstrated benefit.

12.5 Change set D: Engram native orchestration

At the actual host repository, separate simulation construction from runtime advancement. Create one persistent NEST owner with legal live-update support, bounded readout, exact logical time, and explicit outcome classification.

Replace project-to-project Host API calls with NCP role clients. Keep model-generation and experimental-control logic in Engram. Do not duplicate CREBAIN physics, Prisoma storage, Galadriel statistics, or Haldir policy inside Engram to avoid integration work.

The coupled-run owner tracks closed pairs and retires uncertain generations. It cannot enable a direct fallback from a gated run. Its terminal record differentiates full completion, known early termination, incomplete capture, and indeterminate suffix.

12.6 Change set E: CREBAIN endpoint

Use `integrations/engram/managed-simulation/` as a semantic baseline and migration location, not as an authorized permanent external bypass. Expose an NCP responder owning the existing body kernel, exact observation layout, application checks, and result retention.

Preserve the one-, two-, and three-entity qualification cases. Add explicit input-source grants where the real-time profile is selected. Add applied-action receipts needed by Prisoma and Haldir. Keep simulator authority separate from any physical application surface.

12.7 Change set F: Prisoma capture

Migrate the historical `crates/ncp-observer/` path using the causal semantics in `crates/engram-managed-observer/src/observer.rs`. Do not just repin the dependency and call it done.

Introduce a generic complete-run capture path independent of PID eligibility. Add whole-step omission detection, bounded joins, exact duplicate/conflict handling, attachment budgets, result/application distinction, and terminal completeness. All Engram-facing runtime access uses NCP.

12.8 Change set G: Galadriel and optional Haldir

First inventory Galadriel's actual source and schemas on the pinned checkout; this report does not pretend that step is complete. Define the minimum evidence adapter and bounded assessment extension. Qualify monitoring and online-guard modes separately.

For Haldir, inspect its existing contracts, gate, historical NCP adapter, transport, and evidence code. Migrate typed intent and advisory admission to NCP extensions, construct native commands under Haldir-only authority, and reconcile CREBAIN dispositions. Keep direct and gated modes mutually exclusive. Do not claim physical readiness from the simulation tests.

12.9 Change set H: independent conformance, distribution, and release evidence

Build each consumer outside the NCP workspace using intended packaged artifacts. Workspace inheritance can conceal missing patches, features, schema files, or dependency assumptions. Use an independent decoder/validator for conformance; an FFI call back into the same Rust implementation is not independent agreement.

Retain exact inputs, expected outcomes, observed outputs, classification of failures, timings, resource evidence, and package identities. A release receipt should identify which role and profile passed, not merely which repository produced a green CI badge.

12.10 Definition of done

A clean installation must run the declared persistent-NEST/CREBAIN experiment through NCP only, preserve the declared numerical and causal semantics, record interpretable Prisoma evidence, supply valid Galadriel assessments or explicit abstention, and enforce the selected direct or gated authority mode. It must contain the specified failures and meet the published resource and latency target on a named configuration.

No integration claim is complete while an undocumented Host API, raw tap, shared store, or direct import can control the runtime. No “verified” claim is complete without naming what was checked and what remained outside the model.

13. Prior-audit coverage and claim ledger

This ledger maps all substantive themes of the previous report into the standalone handoff. “Source confirmed” means the public source or documentation supports the narrow observation; it does not mean the actual application was reproduced.

Prior audit theme	Status and location
Candidate not yet a qualified complete loop	Source/documentation supported; Sections 1 and 3.
Existing managed Host API path	Supported; now explicitly a migration source, not a permitted delivered integration; Sections 3 and 6.
Sensor decimation mistaken for loss	Source-derived counterexample and executed arithmetic; Sections 4.1 and 9.2.
Freshness starts at consumption	Source confirmed, guarantee narrowed to post-ingress age; Sections 4.2 and 7.5.
Missing codec values become midpoints	Source confirmed; duplicate-mapping claim narrowed; Section 4.3.
Sparse components and unit overwrite	Source confirmed; preparation requirements in Sections 4.3 and 5.
Idempotency	Source confirmed; duplicate-execution inference explicitly rejected; Sections 4.5 and 7.
pending/eviction/replay order	Ledger-supported; formula derived and finite-checked; Sections 4.8 and 9.1.
Horizon formula and validator drift	Source confirmed; Section 4.9.
Advisory contract hash called a match	
Secure ingress principal gap	Reviewed-integration limitation, not universal transport claim; Section 4.10.
Broad plane grants	Source confirmed; exact role/operation scope proposed; Sections 4.10 and 6.
Duplicate parse/serialize and copying	Source patterns confirmed, unmeasured cost; Section 5.
Initial writer reservation	Source confirmed, memory/performance interpretation narrowed; Sections 2 and 5.
Repeated profile validation	Source confirmed, immutable-preparation recommendation; Section 5.
Observer scheduling and byte budgets	Source patterns plus conditional capacity calculation; Section 5.
Lockstep versus sampled control	Proposed design, expanded in Sections 6 and 7.
One owner and one outstanding step	Proposed invariant with abstract checks; Sections 7 and 9.7.
Unknown mutation retires state	Expanded to the coupled generation and closed prefix; Section 7.4.
Do not pipeline causal dependencies	Preserved; Sections 7.2 and 9.4–9.5.
Local/remote/physical profiles	Preserved with NCP-only semantics; Sections 5 and 6.
NEST preparation and live changes	Corrected; persistent state with qualified update matrix; Sections 2 and 8.1.
NEST timebase and event windows	Derived timebase plus required backend tests; Sections 8.1 and 9.3.
Bounded readout and CPU placement	Preserved, unmeasured; Sections 8.1 and 10.

Prior audit theme	Status and location
CREBAIN units, per-entity semantics, direct baseline	Preserved; direct baseline test-only; Sections 3.1 and 8.2.
Command proposal is not application	Preserved and made a required disposition; Sections 7 and 8.
Prisoma whole-step omission and scientific limits	Preserved; Section 8.3.
Recorder impossibility tradeoff	Proven under an explicit queue model; Section 9.8.
Slop as terminology/implementation gap	Preserved without claiming fake code; Section 11.
Full latency versus partial timer	Source confirmed; Section 10.1.
Payload and 200-microsecond target	Arithmetic checked; target remains hypothetical; Section 10.2.
Paired methodology and loaded benchmark	Preserved; Sections 10.3-10.4.
Native slice, installed artifacts, independent peers	Preserved; Section 12 and Appendix A.
Original 50 acceptance tests	All included, with clarified wording; Appendix A.

New material includes the small-span interpolation counterexample, thread-origin migration issue, exact large-gap update, one-step stability proof, source-age qualification, explicit NCP-only enforcement, Galadriel/Haldir role design, coupled closed-prefix semantics, and executable mathematical checks. Proposed mechanisms are not represented as implemented fixes.

Appendix A. Seventy acceptance cases

Status: proposed, not executed against the repositories during this review. The original 50 cases are retained below. Cases 51–70 cover the expanded NCP-only boundary and new findings. The finite mathematical checks in Appendix C do not turn these integration cases into passing results.

Run against recorded immutable commits and installed artifacts. Retain exact input, initial state, output, final state, rejection/outcome classification, and relevant timing/resource measurements. Record the profile, bindings, NEST build and model identity, seeds, thread/MPI placement, adapter identities, and each case's scope: unit, process boundary, or deployment qualification. An FFI wrapper around the same validator is not an independent implementation.

P0 denotes semantic correctness or containment before trusting a run. P1 denotes a required behavior of the selected deployment/performance profile. P2 denotes broader distribution and interoperability hardening; it is still mandatory when the advertised release includes that scope.

A. Causality and clocks

01 · P0. Drive a known plant observation through neural input, one neural advance, controller output, and one plant advance.

Required: Every returned observation and applied action has the declared step index and causal predecessor. No action is applied a step early or late.

02 · P0. Request a neural/plant cadence that cannot be represented by the selected common integer timebase.

Required: Preparation rejects it; there is no silent rounding or accumulated drift.

03 · P0. Mix timestamps from unrelated monotonic clock domains.

Required: The runtime refuses unsupported age/one-way-latency calculations. Logical time remains independent of wall-clock measurements.

04 · P0. Receive a valid advancing sensor, stall the control loop past its freshness budget, then consume the pending sample.

Required: Original receive time is retained; consuming the frame does not make it fresh again. This bounds post-ingress waiting; test 55 covers body-source-to-application age.

05 · P0. Execute the same prepared deterministic case at different wall-clock pacing rates, without changing logical inputs.

Required: Results satisfy the declared reproducibility policy. Wall-clock pacing does not silently change the numerical experiment.

B. Stream positions and delivery semantics

06 · P0. Deliver every 200 Hz sensor sample while a 50 Hz controller intentionally consumes only the latest.

Required: Ingress loss remains zero. Coalescing is counted separately. No loss-triggered emergency latch is set solely by downsampling.

07 · P0. Inject genuine ingress gaps, duplicates, and bounded reordering separately.

Required: Counters distinguish the cases; duplicates cannot refresh freshness or conceal missing positions. Attribution is not stronger than the evidence.

08 · P0. Use a very large accepted loss-detector threshold and a huge sequence gap.

Required: Configuration is rejected or the bounded-time mathematical update gives the correct restrictive result. No iteration cap silently suppresses the threshold crossing.

09 · P0. Deliver an old generation after reconnect, including an emergency command.

Required: It cannot change the new generation's state. Restart requires a new prepared binding.

10 · P0. Replace an unsent latest-value command while a recorder observes controller proposals.

Required: Candidate identity, transport admission, actual publication, and application remain distinguishable. One published stream position never acquires conflicting committed payloads.

C. Stateful execution and retries**11 · P0. Lose a successful step reply and resend the identical operation identity and payload.**

Required: The retained outcome is returned; the neural simulator and plant are not advanced again.

12 · P0. Reuse an operation identity with changed input bytes or semantic content.

Required: Conflict is rejected before mutation. Canonicalization policy is explicit.

13 · P0. Fill the outcome store while all retained outcomes are still contractually owed.

Required: New mutations are rejected before execution, or an explicitly negotiated acknowledgement releases retention. Required outcomes are not silently evicted.

14 · P0. Query a retained result after the original execution deadline or lease expires.

Required: An authorized result-lookup policy can recover the outcome without granting new mutation authority. Unauthorized callers receive no protected result.

15 · P0. Kill, time out, or cancel a backend after execution starts but before commit can be established.

Required: The result is marked indeterminate and that mutable generation is retired. Cancellation is not reported as rollback. No retry advances uncertain state again.

D. Layout, units, and missingness**16 · P0. Transport a six-component position/velocity observation and a three-component acceleration action.**

Required: Units and frames remain component-correct. Position is not labeled as velocity, and velocity setpoints are not silently treated as acceleration.

17 · P0. Omit a required population readout whose configured command range is [0, 10].

Required: The absence is explicit. It cannot become an apparently observed Active command of 5 merely through midpoint substitution.

18 · P0. Compare an available zero-valued signal, an unavailable signal, an empty spike window, and missing recording evidence.

Required: All four retain their distinct meanings through NCP and Prisoma.

19 · P0. Prepare duplicate, sparse, contradictory-unit, or reordered component mappings.

Required: Invalid layouts are rejected before running. Accepted permutations have explicit immutable mappings and identical interpreted meaning.

20 · P0. Inject non-finite values, overflowing counters, wrong arity, invalid availability padding, and forbidden numeric ranges.

Required: Bounded rejection occurs before state mutation. An invalid frame is not repaired into valid actuation.

E. NEST adapter behavior

21 • P0. Execute many control steps with a prepared persistent NEST network.

Required: The network is built and prepared according to the selected adapter contract, not reset or rebuilt per tick. Neural state persists. A single Prepare across all steps is required only when the qualified update mechanism permits it.

22 • P0. Change each supported stimulus parameter between successive Run calls in the exact selected NEST build.

Required: Measured neuron/device behavior proves the input actually takes effect in the promised interval. Unsupported mutations are rejected at preparation.

23 • P0. Place spikes exactly on observation-window boundaries and, where supported, off-grid timestamps.

Required: Every event is counted once under the declared endpoint convention; precise off-sets are not silently rounded away.

24 • P1. Compare long-run per-step extraction cost for counters and bounded event tails.

Required: Cost and retained memory do not grow with total historical recording length when only a bounded current window is requested. Counter reset semantics are handled explicitly.

25 • P1. Repeat the adapter workload with declared thread/MPI configurations and transport/observer contention.

Required: Reproducibility and deadline claims remain scoped to measured configurations. Simulator workers do not silently consume all service capacity needed for feedback.

F. CREBAIN and controller behavior

26 • P0. Run one-, two-, and three-entity cases through the selected real CREBAIN adapter.

Required: Roster ordering, per-entity state, and action routing remain exact. No cross-entity substitution occurs.

27 • P0. Compare a direct closed-loop baseline against the NCP-bound loop using the same controller, inputs, seeds, and schedule.

Required: Trajectory and event differences satisfy the declared numerical policy; transport introduction does not change the experiment unintentionally. The direct oracle is an isolated test target, unreachable as an alternative path in the distributed runtime.

28 • P0. Remove one entity's observation while others remain valid.

Required: Missingness and fallback are per entity. Valid peers are neither fabricated nor inadvertently discarded.

29 • P0. Trigger HOLD and emergency behavior for every declared plant profile.

Required: The runtime applies the profile-specific behavior. Zero acceleration is not represented as a universal physical stop.

30 • P0. Expire or revoke command authority at the actual application boundary.

Required: An accepted or queued command cannot bypass current application-time authority and freshness checks. Simulator-only credentials cannot authorize physical actuation.

G. Prisoma and evidence integrity

31 • P0. Record a successful step end to end.

Required: Evidence binds input snapshot, neural request/result, controller proposal, admitted/applied action, output snapshot, and outcome without conflating them.

32 • P0. Remove every event belonging to one whole step.

Required: The terminal plan/progress record reveals incomplete capture. A self-consistent retained subset is not declared complete.

33 · P0. Deliver observation and command events out of order, twice, and with conflicting payloads.

Required: Bounded joins, duplicate handling, and conflicts are explicit; arrival order does not fabricate causality.

34 · P0. Exceed recording capacity under the lossless logical-time profile.

Required: The host pauses at a defined boundary or terminates the capture/run according to policy. It does not silently lose records while claiming completeness.

35 · P1. Exceed recording capacity under the real-time best-effort profile.

Required: Feedback stays within its selected resource policy; drops and affected completeness claims are explicitly recorded. Recorder lag is not mislabeled as simulator or network failure.

H. Resource bounds and scheduling

36 · P1. Send maximum-size accepted observation messages to all permitted subscribers.

Required: Aggregate bytes, not only item counts, stay within declared reservations including retained buffer ownership.

37 · P1. Use a slow, continuously busy, panicking, and non-returning observer callback.

Required: Observer work cannot monopolize the control executor. Containment and teardown policy are demonstrated, not inferred from an async function name.

38 · P1. Saturate lifecycle/backend requests with slow or stuck operations.

Required: Admission remains bounded. Per-backend mutation is serialized, and terminal cleanup/containment has a defined path.

39 · P1. Instrument a warmed-up small-frame step with allocation, copy, serialization, and profile-hash counters.

Required: The selected fast-path budget is met; unexpected repeated work is a test failure rather than an undocumented optimization opportunity.

40 · P1. Repeatedly prepare, run, cancel, finish, and disconnect sessions.

Required: Threads, tasks, subscribers, handles, memory, and generation state return to their declared baseline or bounded retained state.

I. Transport identity and conformance

41 · P0. Use a valid certificate or local capability for the wrong principal, route, operation, role, or session kind.

Required: Receiver-owned authorization rejects it; a payload identity claim cannot substitute for authenticated origin.

42 · P0. Rotate/revoke a principal while messages are queued.

Required: Each admission is tied to a defined immutable policy generation, and the selected revocation boundary is enforced consistently.

43 · P0. Request an unavailable secure profile or attempt downgrade to an unauthenticated binding.

Required: Connection/admission fails closed. Local research transport is selected explicitly, never as a silent fallback.

44 · P1. Run horizon vectors at exact expiry, beyond the maximum TTL, and with a non-finite ratio.

Required: Independent implementations produce identical acceptance and executable-window decisions.

45 · P1. Omit or alter the selected semantic/profile identity during preparation.

Required: A prepared strict session cannot form from advisory compatibility alone. Optional-feature negotiation does not change already-prepared meaning.

J. Performance evidence and release usability**46 · P1. Measure complete sample-to-application latency and each internal stage.**

Required: Report p50, p99, p99.9, maximum observed, deadlines, and failures. A timer ending before serialization or publication is not called closed-loop latency.

47 · P1. Compare matched direct and NCP runs, including instrumentation overhead.

Required: Attribute transport/integration cost with a justified paired methodology; do not subtract unrelated percentile summaries.

48 · P1. Exercise scheduled arrivals during stalls, background load, and slow recording.

Required: Missed work and delay are counted against the intended schedule. The benchmark does not hide stalls by generating less offered work.

49 · P2. Install and build CREBAIN/Prisoma consumers outside the NCP workspace, from the intended published artifacts.

Required: Dependency pins, patches, features, names, schemas, and bindings work without undocumented workspace inheritance.

50 · P2. Reproduce the complete chosen-profile run from clean artifacts and retained inputs.

Required: Another operator obtains the declared behavior and scoped performance results. Unsupported profiles remain visibly unsupported.

K. Expanded boundary, numerical, and recovery tests**51 · P0. Run Engram against all approved ecosystem peers with only NCP endpoints enabled.**

Required: The full declared workflow succeeds without Host API, custom HTTP/IPC, direct engine import, shared storage, or file-polling integration. NEST remains private to its owner.

52 · P0. Attempt each forbidden cross-project path deliberately.

Required: Package policy and runtime sandboxing reject the bypass. Log the rejection without granting access through a helper process or broad inherited credential.

53 · P1. Move a control-loop object between execution threads with different thread-local elapsed origins.

Required: Time remains in one defined domain or migration is disallowed explicitly. A valid move does not create false expiry or a spurious clock-rewind hold.

54 · P0. Encode endpoints of a finite positive range narrower than machine epsilon near one.

Required: The upper endpoint maps to the declared upper output, or preparation rejects the explicitly unsupported scale. Unit changes do not silently collapse valid ranges.

55 · P0. Delay a body-originated source grant by 15 ms in each direction with a 20 ms budget.

Required: Final application rejects expiry relative to the original body event. No receiver or relay restarts the deadline.

56 · P0. Return a grant from a different source, generation, authority term, or evicted record.

Required: No application occurs from a guessed or substituted reference. A receiver never falls back to the newest grant.

57 · P0. Cut execution after each neural/body commit and before each receipt or pair closure.

Required: The maximal known closed prefix is preserved. The uncertain suffix is explicitly classified and no dependent step advances an unresolved coupled generation.

58 · P0. Lose a committed response, then retrieve it after the result was acknowledged or while still retained.

Required: Retained exact payload and receipt are replayed when owed. After payload release, high-water state prevents re-execution and the declared unavailable outcome is returned.

59 · P0. Force result serialization or byte-reservation failure after a backend mutation.

Required: The outcome is never labeled rejected-before-execution. Required containment or retirement occurs and no alternate operation ID retries the uncertain effect.

60 · P0. Select Haldir-gated mode, then attempt direct Engram command and gate-timeout fallback.

Required: CREBAIN rejects Engram as commander. Gate failure follows the prepared restrictive policy; it does not activate a parallel direct route.

61 · P0. Feed the Galadriel adapter missing uncertainty, wrong units, invalid covariance, or an unsupported detector profile.

Required: The actual selected detector contract rejects or abstains explicitly. It does not fabricate uncertainty or label absent assessment nominal.

62 · P0. Submit a favorable or forged Galadriel assessment without independently granted action authority.

Required: No lease, permission widening, final command, or reset capability is created. Advisory composition is monotone with respect to existing permission.

63 · P0. Compare asynchronous monitoring and step-bound online-guard profiles under reordered assessments.

Required: Monitoring cannot affect state. The guard consumes only its declared exact-step dependency and has explicit missing/stale/abstention policy.

64 · P0. Give a private worker, presentation panel, or extension parser an attempted out-of-role request.

Required: It has neither credentials nor inherited handles that allow role escalation, command publication, or direct access to another project's mutable state.

65 · P2. Rebuild the real Engram host and all adapters from the recorded repository identities.

Required: The executable host, not a placeholder repository, is used. Dirty or mismatched builds and unrecorded private patches cannot receive a qualification receipt.

66 · P0. Remove or conflict the terminal run record while all retained individual rows remain valid.

Required: Capture completeness is unresolved or invalid, not inferred from the apparently consistent subset. Planned versus completed counts stay distinct.

67 · P1. Saturate observer and result reservations before the next lossless logical step.

Required: Admission pauses or rejects before neural/body mutation. Reservation release after acknowledgement is bounded and cannot be counted twice.

68 · P0. Make a GUI projection stale, altered, or disconnected while the NCP data path remains active.

Required: No agent/controller consumes the presentation projection as authoritative input. UI status cannot refresh source grants or authority.

69 · P0. Request oversized simulation creation or embedded executable code through an external data field.

Required: The bounded model-creation contract rejects unsupported resource requests and code execution. Cancellation/cleanup does not leak an uncontrolled worker.

70 · P1. Run independent decoders against exact integers, changed semantic identities, unknown enums, duplicate keys, availability padding, and horizon boundaries.

Required: Every advertised public path agrees with the chosen contract. Unknown diagnostic data cannot become executable control semantics; private dialects fail qualification.

Acceptance gate order

First establish the semantic and enabled authority boundaries with failing-then-passing native regressions. Next run a native persistent-NEST/CREBAIN slice with Prisoma capture over the selected NCP binding. Add Galadriel monitoring and optional gated-mode qualification. Then qualify resource/performance behavior and clean installed-artifact reproduction.

A scoped local simulator release need not wait for every remote or physical profile. It must not claim those profiles, lossless capture, crash resume, or real-time guarantees merely because one local fixture passes. An unavailable required test is recorded as not run and blocks its associated claim.

Appendix B. Execution instructions for the implementation agent

Treat Sections 6–8 as target requirements, Section 4 as source-derived findings to reproduce on pinned code, and Appendix A as unexecuted acceptance work. Do not treat a proposal as an existing public API.

Start by capturing immutable source identities and resolving the actual Engram host repository. Recheck current symbols and write the minimal native counterexamples. Preserve useful existing behavior and avoid a wholesale rewrite that discards validation. Reconcile protocol changes in NCP first, then consume its versioned artifacts across repositories.

Complete a single NCP-only direct simulation slice before broadening the deployment surface. Keep Haldir optional but mutually exclusive with direct authority when selected. Make Galadriel's evidence requirements explicit and Prisoma's generic capture independent of analysis eligibility. No legacy runtime bypass remains enabled in the delivered package.

For each merged change retain: the behavior being changed; the exact prior and new contract; a regression; independent expected output; scope of executed tests; measured resource impact where relevant; and unresolved limitations. A regenerated receipt or a green wrapper test is not independent proof.

The final agent handoff must include patches, source/package lock identities, commands actually run, exact test outcomes, a complete sample run, fault-injection results, and the measured latency/resource distribution. Keep unexecuted items visible. Do not fill missing evidence with invented logs, arbitrary confidence scores, or claims of universal verification.

Appendix C. Reproducible mathematical verification program

The complete program is included so the document stands alone. It requires Python 3.10 or later and only the standard library. Save the code as `verify_design.py` and run:

```
python3 verify_design.py --output verification_results.json
```

The program was executed with Python 3.13.5 for this report. It uses exact fractions for the principal arithmetic checks, floating-point roots for the illustrative stability sweep, and finite abstract state machines for ownership/retirement rules. It does not import or execute NCP, NEST, CREBAIN, Prisoma, Galadriel, or Haldir. Repeated invariant assertions are not independent experimental trials.

```
#!/usr/bin/env python3
"""Executable mathematical checks for the NCP closed-loop design report.

Standard library only. These are exact-arithmetic property checks and small
abstract models, NOT tests of the Rust repositories, NEST, or a transport.
Run: python3 verify_design.py --output verification_results.json
Assertions use explicit exceptions so python -O cannot disable the checks.
"""

from __future__ import annotations

import argparse
import cmath
import itertools
import json
import math
import platform
from fractions import Fraction as F
from pathlib import Path
from typing import Any

COUNTS: dict[str, int] = {}
EXAMPLES: dict[str, Any] = {}

def check(group: str, predicate: bool) -> None:
    if not predicate:
        raise AssertionError(f"Property failed: {group}")
    COUNTS[group] = COUNTS.get(group, 0) + 1

def ceil_q(x: F) -> int:
    return -((-x.numerator) // x.denominator)

def strict_horizon(ttl: F, dt: F, cap: int) -> int:
    if ttl <= 0 or dt <= 0 or cap < 0:
        raise ValueError("positive times and nonnegative count cap required")
    return min(cap, max(0, ceil_q(ttl / dt) - 1))

def cusum_events(start: F, misses: int, reference: F) -> F:
    s = start
    for _ in range(misses):
        s = max(F(0), s + 1 - reference)
    return max(F(0), s - reference)

def cusum_closed(start: F, misses: int, reference: F) -> F:
    return max(F(0), start + misses * (1 - reference) - reference)

def rational_quantum(times: list[F]) -> F:
    if not times or any(x <= 0 for x in times):
        raise ValueError("positive nonempty time list required")
    denominator = math.lcm(*(x.denominator for x in times))
    integers = [x.numerator * (denominator // x.denominator) for x in times]
    return F(math.gcd(*integers), denominator)

def main(output: Path) -> None:
```

```

# Independent oracle: explicitly enumerate all executable future positions.
for tn, td, dn, dd, cap in itertools.product(
    range(1, 13), range(1, 5), range(1, 7), range(1, 4), (0, 1, 3, 10)
):
    ttl, dt = F(tn, td), F(dn, dd)
    expected = sum(j * dt < ttl for j in range(1, cap + 1))
    check("strict_horizon", strict_horizon(ttl, dt, cap) == expected)
for ttl, dt in itertools.product(range(1, 101), range(1, 31)):
    check("integer_horizon", (ttl - 1) // dt == strict_horizon(F(ttl), F(dt), 1000))

# Exact algebraic batching, including gaps greater than the current cap.
for s, m, r in itertools.product(
    (F(0), F(1, 10), F(1), F(5)),
    (0, 1, 2, 3, 17, 255, 256, 257, 1000),
    (F(0), F(1, 20), F(1, 2), F(99, 100)),
):
    check("cusum_batch", cusum_events(s, m, r) == cusum_closed(s, m, r))
s1 = cusum_closed(F(0), 3, F(1, 20))
s2 = cusum_closed(s1, 3, F(1, 20))
check("decimation_counterexample", s1 == F(14, 5) and s2 == F(28, 5) and s2 >= 5)
full = cusum_closed(F(0), 1000, F(1, 20))
capped = cusum_closed(F(0), 256, F(1, 20))
check("gap_cap_counterexample", capped < 300 < full)
EXAMPLES["cusum"] = {
    "decimated_consumption": [float(s1), float(s2)],
    "gap_1000_exact": float(full), "gap_256_capped": float(capped),
    "threshold": 300,
}

# Small but strictly positive valid binary64 span: EPSILON is not a scale.
span = 2.0 ** -60
epsilon = 2.0 ** -52
old_output = 0.0 if abs(span) < epsilon else 200.0 * (span / span)
mathematical_output = 200.0 * (span / span)
check("small_span_counterexample", span > 0 and old_output == 0 and mathematical_output == 200)
EXAMPLES["small_span"] = {"span": span, "epsilon": epsilon,
    "old_output": old_output, "required_endpoint": mathematical_output}

# A live object moved between threads need not retain a thread-local origin.
absolute_t1, origin_a, absolute_t2, origin_b = F(100), F(0), F(101), F(100)
check("thread_origin_counterexample", absolute_t2 > absolute_t1 and
    absolute_t2 - origin_b < absolute_t1 - origin_a)
EXAMPLES["thread_origin"] = {"absolute_samples": [100, 101],
    "thread_local_elapsed_samples": [100, 1]}

# Rational common tick construction and exact accumulated position.
pool = (F(1, 10000), F(1, 1000), F(1, 100), F(1, 50), F(1, 60), F(1, 30))
for pair in itertools.combinations(pool, 2):
    q = rational_quantum(list(pair))
    ticks = [x / q for x in pair]
    check("rational_timebase", all(t.denominator == 1 for t in ticks))
    check("rational_timebase", math.gcd*(t.numerator for t in ticks) == 1)
q = rational_quantum([F(1, 10000), F(1, 50)])
check("rational_timebase", 1_000_000 * 200 * q == F(20000))
EXAMPLES["timebase"] = {"quantum_seconds": str(q), "ticks_per_20ms": 200}

# A one-step transport phase error can change stability even at zero RTT.
stable_now, stable_delayed = [], []
for n in range(-10, 31):
    a = n / 10.0
    immediate = abs(1 - a) < 1 - 1e-10
    root = cmath.sqrt(1 - 4 * a)
    delayed = max(abs((1 + root) / 2), abs((1 - root) / 2)) < 1 - 1e-10
    check("delay_stability", immediate == (0 < a < 2))
    check("delay_stability", delayed == (0 < a < 1))
    if immediate:
        stable_now.append(a)
    if delayed:
        stable_delayed.append(a)
check("delay_stability", F(3, 2) > 1 and abs(1 - F(3, 2)) < 1)
EXAMPLES["delay_stability"] = {"a": 1.5, "no_delay_pole": -0.5,
    "one_step_delay_pole_modulus": math.sqrt(1.5)}

# Body-originated grant checks: total elapsed round trip, no relay renewal.

```

```

for outbound, computation, inbound, budget in itertools.product(
    range(8), range(8), range(8), range(1, 12)
):
    sampled_at = 100
    application = sampled_at + outbound + computation + inbound
    deadline = sampled_at + budget
    accepted = application < deadline
    check("source_grant", accepted == (outbound + computation + inbound < budget))
check("freshness_counterexample", 100 + 15 + 15 >= 100 + 20 and 115 + 15 < 115 + 20)
EXAMPLES["freshness"] = {"budget_ms": 20, "outbound_ms": 15,
    "return_ms": 15, "source_grant_accepts": False,
    "receiver_restarted_ttl_accepts": True}

# Finite deterministic equivalence example; wall pacing is not model input.
def transition(state: tuple[F, F], obs: F) -> tuple[tuple[F, F], F]:
    plant, neural = state
    neural_next = neural / 2 + obs / 4
    action = -neural_next
    return (plant + action / 10, neural_next), action
direct, adapted = (F(1), F(0)), (F(1), F(0))
for step in range(200):
    direct, action_d = transition(direct, direct[0])
    payload = (step, str(adapted[0]))
    decoded_observation = F(payload[1])
    adapted, action_a = transition(adapted, decoded_observation)
    check("deterministic_equivalence_model", direct == adapted and action_d == action_a)

# Paired advancement: a tiny state model, not an implementation proof.
events = ("neural_commit", "plant_commit", "close", "uncertain", "restart")
for history in itertools.product(events, repeat=5):
    phase, retired = "ready", False
    neural_count = plant_count = closed_count = 0
    frozen = None
    for event in history:
        if retired:
            # Same-generation restart never reactivates this object.
            check("paired_prefix_model", frozen ==
                (phase, neural_count, plant_count, closed_count))
            continue
        if event == "uncertain":
            retired = True
            frozen = (phase, neural_count, plant_count, closed_count)
        elif event == "neural_commit" and phase == "ready":
            neural_count += 1
            phase = "neural"
        elif event == "plant_commit" and phase == "neural":
            plant_count += 1
            phase = "plant"
        elif event == "close" and phase == "plant":
            closed_count += 1
            phase = "ready"
        check("paired_prefix_model", closed_count <= plant_count <= neural_count)
        check("paired_prefix_model", neural_count - closed_count <= 1)
    if retired:
        check("paired_prefix_model", frozen ==
            (phase, neural_count, plant_count, closed_count))

# Finite abstract operation owner: retained result + high water after ACK.
events = ("new", "retry", "commit", "ack", "uncertain")
for history in itertools.product(events, repeat=6):
    state, executions, committed, highwater = "empty", 0, False, 0
    for event in history:
        if event == "new" and state == "empty" and highwater == 0:
            state, executions = "pending", executions + 1
        elif event == "commit" and state == "pending":
            state, committed, highwater = "retained", True, 1
        elif event == "ack" and state == "retained":
            state = "acknowledged"
        elif event == "uncertain" and state == "pending":
            state = "retired"
        # 'retry' only replays/returns pending/unknown; never executes.
        check("at_most_once_model", executions <= 1)
        check("at_most_once_model", not committed or highwater == 1)

```

```

# Innovation quadratic form: exact 2x2 Cholesky example, not calibration.
# L=[[2,0],[1,3]], S=L*L^T=[[4,2],[2,10]], det(S)=36.
for r0, r1 in itertools.product(range(-5, 6), repeat=2):
    v0 = F(r0, 2)
    v1 = (F(r1) - v0) / 3
    direct_q = F(10*r0*r0 - 4*r0*r1 + 4*r1*r1, 36)
    check("innovation_quadratic", direct_q == v0*v0 + v1*v1)
    check("innovation_quadratic", direct_q >= 0)

# Exact recurrence at its error bound, including L=0 and L=1.
for lip in (F(0), F(1, 2), F(1), F(6, 5)):
    error, eps = F(0), F(1, 100)
    for k in range(1, 31):
        error = lip * error + eps
        bound = eps * sum((lip**i for i in range(k)), F(0))
        check("error_propagation", error == bound)

# Equal marginal variance: explicitly sum all covariance entries.
for n, rho in itertools.product(range(2, 21), (F(0), F(1, 10), F(1, 2))):
    total = sum((F(1) if i == j else rho
                  for i in range(n) for j in range(n)), F(0))
    check("correlated_count_variance", total == n * (1 + (n-1)*rho))

# An advisory restriction can only shrink independently granted permission.
universe = set(range(5))
subsets = [{i for i in universe if bits & (1 << i)} for bits in range(32)]
for base, advisory in itertools.product(subsets, repeat=2):
    check("authority_monotonicity", (base & advisory) <= base)

# Fluid arrival/service envelope, conditional on c>=r and FIFO service.
for burst, rate, capacity, latency in itertools.product(
    (F(1), F(3)), (F(0), F(1), F(2)), (F(2), F(4)), (F(0), F(1), F(3))
):
    if capacity < rate:
        continue
    delay_bound = latency + burst / capacity
    backlog_bound = burst + rate * latency
    for i in range(101):
        t = F(i, 10)
        alpha = burst + rate * t
        beta = capacity * max(F(0), t - latency)
        check("queue_envelope_model", alpha - beta <= backlog_bound)
        service_shifted = capacity * max(F(0), t + delay_bound - latency)
        check("queue_envelope_model", service_shifted >= alpha)

# Poisson assumptions are explicit; these are formulas, not observed spikes.
for neurons in (8, 100):
    rate, window = F(20), F(1, 50)
    expected_count = neurons * rate * window
    variance_rate_estimate = rate / (neurons * window)
    check("poisson_rate_formula", variance_rate_estimate / rate ** 2 == 1 / expected_count)
EXAMPLES["poisson_cv"] = {"N100_r20_window20ms": 1 / math.sqrt(40),
                          "N8_r20_window20ms": 1 / math.sqrt(3.2)}
check("payload_arithmetic", 3 * (6 + 3) * 8 == 216)
check("payload_arithmetic", 216 * 50 == 10800)
check("payload_arithmetic", 64 * 1024 * 1024 == 67108864)
check("payload_arithmetic", F(20, 1000) / 100 == F(200, 1000000))

result = {
    "status": "PASS",
    "scope": "Exact-arithmetic checks, numerical examples, and abstract finite models only; no
    ↪ repository or NEST execution.",
    "python": platform.python_version(),
    "group_count": len(COUNTS),
    "assertion_count": sum(COUNTS.values()),
    "groups": COUNTS,
    "examples": EXAMPLES,
}
output.parent.mkdir(parents=True, exist_ok=True)
output.write_text(json.dumps(result, indent=2) + "\n", encoding="utf-8")
print(json.dumps({k: result[k] for k in ("status", "group_count", "assertion_count", "scope")},
    ↪ indent=2))

```

```
if __name__ == "__main__":  
    parser = argparse.ArgumentParser(description=__doc__)  
    parser.add_argument("--output", type=Path, default=Path("verification_results.json"))  
    args = parser.parse_args()  
    main(args.output)
```

Appendix D. Recorded verification output

This is the actual output file produced by the included program. All 21 groups passed. The assertion count is a program-execution count over finite examples and histories; it is not the number of repository tests, proofs, audited claims, or independent replications.

```
{
  "status": "PASS",
  "scope": "Exact-arithmetic checks, numerical examples, and abstract finite models only; no repository
↪ or NEST execution.",
  "python": "3.13.5",
  "group_count": 21,
  "assertion_count": 237003,
  "groups": {
    "strict_horizon": 3456,
    "integer_horizon": 3000,
    "cusum_batch": 144,
    "decimation_counterexample": 1,
    "gap_cap_counterexample": 1,
    "small_span_counterexample": 1,
    "thread_origin_counterexample": 1,
    "rational_timebase": 31,
    "delay_stability": 83,
    "source_grant": 5632,
    "freshness_counterexample": 1,
    "deterministic_equivalence_model": 200,
    "paired_prefix_model": 28231,
    "at_most_once_model": 187500,
    "innovation_quadratic": 242,
    "error_propagation": 120,
    "correlated_count_variance": 57,
    "authority_monotonicity": 1024,
    "queue_envelope_model": 7272,
    "poisson_rate_formula": 2,
    "payload_arithmetic": 4
  },
  "examples": {
    "cusum": {
      "decimated_consumption": [
        2.8,
        5.6
      ],
      "gap_1000_exact": 949.95,
      "gap_256_capped": 243.15,
      "threshold": 300
    },
    "small_span": {
      "span": 8.673617379884035e-19,
      "epsilon": 2.220446049250313e-16,
      "old_output": 0.0,
      "required_endpoint": 200.0
    },
    "thread_origin": {
      "absolute_samples": [
        100,
        101
      ],
      "thread_local_elapsed_samples": [
        100,
        1
      ]
    },
    "timebase": {
      "quantum_seconds": "1/10000",
      "ticks_per_20ms": 200
    },
    "delay_stability": {
      "a": 1.5,
      "no_delay_pole": -0.5,
      "one_step_delay_pole_modulus": 1.224744871391589
    },
    "freshness": {
      "budget_ms": 20,

```

```
    "outbound_ms": 15,  
    "return_ms": 15,  
    "source_grant_accepts": false,  
    "receiver_restarted_ttl_accepts": true  
  },  
  "poisson_cv": {  
    "N100_r20_window20ms": 0.15811388300841897,  
    "N8_r20_window20ms": 0.5590169943749475  
  }  
}
```

Appendix E. Sources, retrieval scope, and evidence discipline

Public sources were retrieved on September 5, 2026. GitHub main URLs and NEST's stable alias are mutable. This report contains no independently verified current commit lock or downloaded source archive. The implementation agent must replace these references with exact checked-out commits and artifact hashes for native qualification.

Source observations are paraphrased and identified by file and symbol rather than unreliable web-rendered line numbers. Repository documentation is evidence of what the project declares, not independent reproduction of its tests. Mathematical arguments and proposed requirements are identified separately in the report.

S01 — NCP repository and README. Candidate status and distinction between implemented and proposed surfaces.

<https://github.com/sepahead/NCP>

S02 — NCP `ncp-core/src/transport.rs`. `NeuroControlLoop::tick`, latest sensor consumption, clock helper, local admission, and timer scope.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/transport.rs>

S03 — NCP `ncp-core/src/resilience.rs`. `LinkMonitor` recurrence, defaults, bounded gap work, and reconciliation storage.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/resilience.rs>

S04 — NCP `ncp-core/src/codec.rs`. `validate_range`, `lerp`, `CodecSpec` validation, missing mappings, sparse output, and units.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/codec.rs>

S05 — NCP `ncp-core/src/idempotency.rs`. `Pending/terminal` retention, begin ordering, receipt replay, and unknown outcomes.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/idempotency.rs>

S06 — NCP `ncp-core/src/messages.rs`. `ContractStatus`, `verify_contract`, bounded typed decoding, and explicit wire admission.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/messages.rs>

S07 — NCP `ncp-core/src/bounded_json.rs`. Resource bounds, duplicate-key rejection, and `CappedJsonWriter` reservation.

https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/bounded_json.rs

S08 — NCP `ncp-zenoh/src/lib.rs`. `ZenohControlTransport`, typed/raw handoff, serialization, publication, and observation workers.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-zenoh/src/lib.rs>

S09 — NCP `ncp-core/src/security.rs`. `PrincipalGrant` and deployed identity/permission structures.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/security.rs>

S10 — NCP `KNOWN_LIMITATIONS.md`. Self-reported horizon parity, transport, native-consumer, performance, and packaging limitations.

https://github.com/sepahead/NCP/raw/refs/heads/main/KNOWN_LIMITATIONS.md

S11 — NCP proposed low-overhead architecture. Proposed prepared execution, source freshness, consumer roles, extension isolation, and direct/gated authority. Not implementation qualification.

https://github.com/sepahead/NCP/raw/refs/heads/main/docs/implementation/NCP_1_0_LOW_OVERHEAD_ARCHITECTURE.md

S12 — NCP authenticated-ingress feasibility study. Reviewed transport-principal feasibility and unqualified secure-path alternatives.

<https://github.com/sepahead/NCP/raw/refs/heads/main/docs/research/authenticated-ingress-feasibility.md>

S13 — CREBAIN managed-simulation README. Host API boundary, exact 20 ms profile, vector semantics, local execution contract, and documented qualification workflow.

<https://github.com/sepahead/crebain/blob/main/integrations/engram/managed-simulation/README.md>

S14 — Prisoma historical NCP observer README. Wire-0.8 and live-integration limits, actual Engram host identity, exact joins, missing scientific variables, and fixture scope.

<https://github.com/sepahead/prisoma/blob/main/crates/ncp-observer/README.md>

S15 — Prisoma managed observer source. Closed-loop timebase and causal request/result/action/runtime binding.

<https://github.com/sepahead/prisoma/raw/refs/heads/main/crates/engram-managed-observer/src/observer.rs>

S16 — NCP PERFORMANCE.md. Separation of microbenchmarks from complete release-bound performance evidence.

<https://github.com/sepahead/NCP/raw/refs/heads/main/PERFORMANCE.md>

S17 — NCP ncp-core/src/plant.rs. Plant-profile validation and `validate_active_command` calling `self.validate`.

<https://github.com/sepahead/NCP/raw/refs/heads/main/ncp-core/src/plant.rs>

S18 — CREBAIN NCP bridge handoff. Historical integration scope and migration boundary.

https://github.com/sepahead/crebain/blob/main/docs/NCP_BRIDGE_HANDOFF.md

S19 — Haldir repository and README. Mission-authorization role, gate-owned commands, historical NCP dependency, and deployment qualification boundary.

<https://github.com/sepahead/haldir>

S20 — Haldir architecture. Ownership, policy, publication, and actor/service separation.

<https://github.com/sepahead/haldir/blob/main/docs/ARCHITECTURE.md>

S21 — Haldir limitations. Live publication/application uncertainty and deployment limitations.

<https://github.com/sepahead/haldir/blob/main/docs/LIMITATIONS.md>

S22 — NEST 3.9 simulation-module documentation. Prepare/Run/Cleanup, Simulate, ResetKernel, legal live changes, and `local_spike_counter` behavior. Version-specific primary documentation.

https://nest-simulator.readthedocs.io/en/v3.9/ref_material/pynest_api/nest.lib.hl_api_simulation.html

S23 — NEST spike_recorder documentation. Recorder time semantics and event-recording behavior; stable alias is mutable.

https://nest-simulator.readthedocs.io/en/stable/models/spike_recorder.html

S24 — NCP NEST_REALTIME.md. Historical NEST experiments and current qualification boundary. Historical measurements are not reproduced as current performance here.

https://github.com/sepahead/NCP/raw/refs/heads/main/NEST_REALTIME.md

Retrieval limitations that must not be hidden

The executable Engram host identified by Prisoma was not available for code inspection in this review. The public Engram placeholder is not a substitute. Galadriel's detailed source could not be fully retrieved; role-level discussion uses the NCP ecosystem proposal and clearly marked target requirements. No claim of a complete Galadriel implementation audit is made. Public web search did not resolve those gaps.

Git clone/download and immutable commit retrieval were unavailable in this environment. Native Rust/TypeScript tests and NEST qualification therefore remain unexecuted. The attached Python results are mathematical and abstract-model checks only. The PDF was generated from this Markdown and checked for rendering and text coverage; that artifact quality check does not enlarge the software-verification claim.